UNIDAD 3 / LAB-01 / INFORME Y EVIDENCIAS

Bitácoras y auditoría del dato en PostgreSQL

Diego Villarreal Lopez · 8 de noviembre de 2026

Entorno de práctica

PostgreSQL 18.6 en Windows, base comercial. Las pruebas se realizaron desde pgAdmin 4 y psql en el mismo equipo.

Resultado de la investigación

El servidor registró accesos, errores y cambios de configuración. Con el nivel mod quedó registrado el cambio de teléfono. La auditoría por fila conservó el RFC anterior y el nuevo del incidente reproducido en la Actividad 12.

La bitácora del servidor también permitió detectar un cambio ejecutado mientras el disparador estaba deshabilitado. Las actividades 16 y 17 conservan la exportación verificada y la cronología que distingue estos casos.

Fuente de la práctica

Guía del Laboratorio 1 de FIC Academy

Actividad 1 - Configuración inicial de la bitácora

Estudiante: Diego Villarreal Lopez

Entorno: PostgreSQL 18.6 en Windows, base `comercial`.

Se consultaron diez parámetros en `pg_settings` antes de modificar la configuración de registro para esta unidad.

Parámetro	Valor observado	Origen mostrado
`log_connections`	Cadena vacía	default
`log_destination`	`stderr`	configuration file
`log_directory`	`log`	default
`log_disconnections`	`off`	default
`log_filename`	`postgresql-%Y-%m-%d_%H%M%S.l`… (recortado en la captura)	default
`log_line_prefix`	`%t`	configuration file
`log_min_error_statement`	`error`	default
`log_statement`	`none`	default
`log_timezone`	`America/Chihuahua`	configuration file
`logging_collector`	`on`	configuration file

Respuestas

1. **¿Se registra quién se conecta?** No se registra rutinariamente quién inicia sesión con éxito. En PostgreSQL 18, el valor vacío de `log_connections` desactiva ese registro. Los fallos de autenticación pueden registrarse de todas formas.

2. **¿Se registran los UPDATE?** Los exitosos no se registran de forma general con `log_statement = none`. Una sentencia que provoque un error sí puede aparecer por `log_min_error_statement = error`.

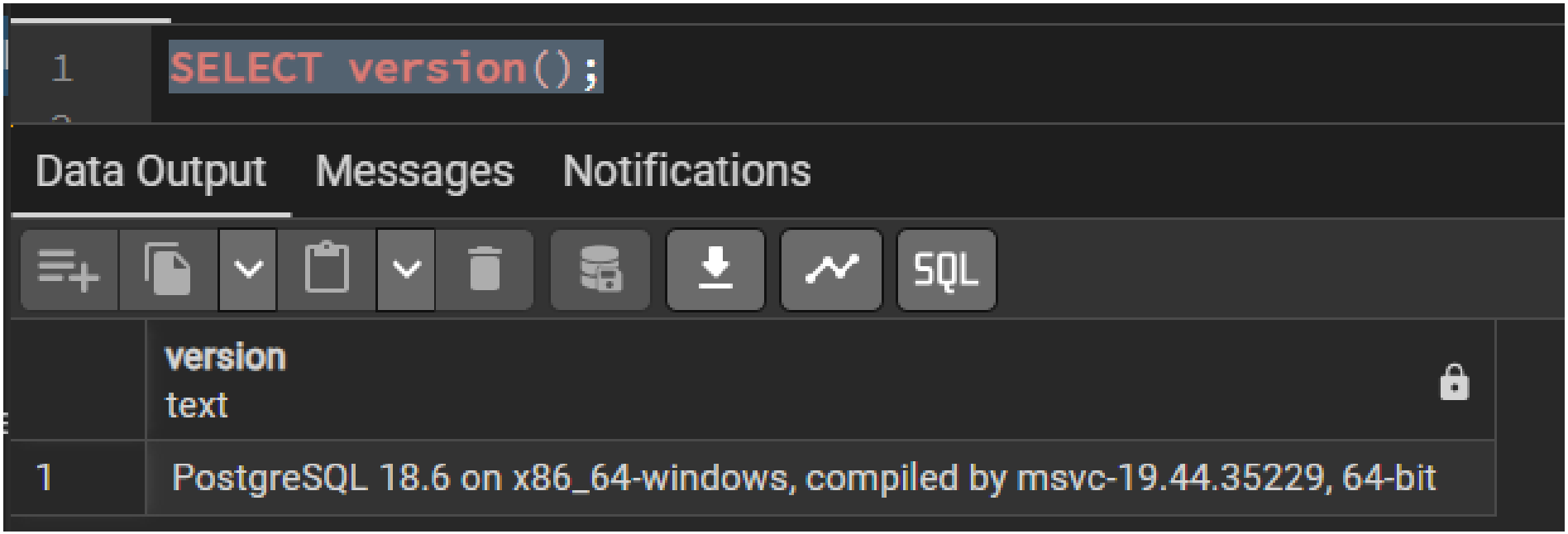
3. **¿El prefijo incluye usuario o IP?** No. `%t` agrega fecha y hora sin milisegundos; el prefijo no incluye campos de usuario ni dirección. Algunos mensajes pueden contener esos datos en su propio texto.

4. **¿En qué zona horaria se registran los eventos?** En `America/Chihuahua`. Identificarla permite ordenar los eventos y compararlos con otros registros que podrían usar UTC u otra zona.

5. **¿Quién decide el destino si el colector está apagado?** Con destino `stderr`, el servicio o mecanismo que inicia PostgreSQL determina cómo se captura esa salida. En este servidor el colector está **activado**, por lo que PostgreSQL recoge la salida y escribe sus archivos de bitácora.

Referencia: [PostgreSQL 18 - Registro de errores y eventos](https://www.postgresql.org/docs/18/runtime-config-logging.html).

Captura 01. Versión del servidor



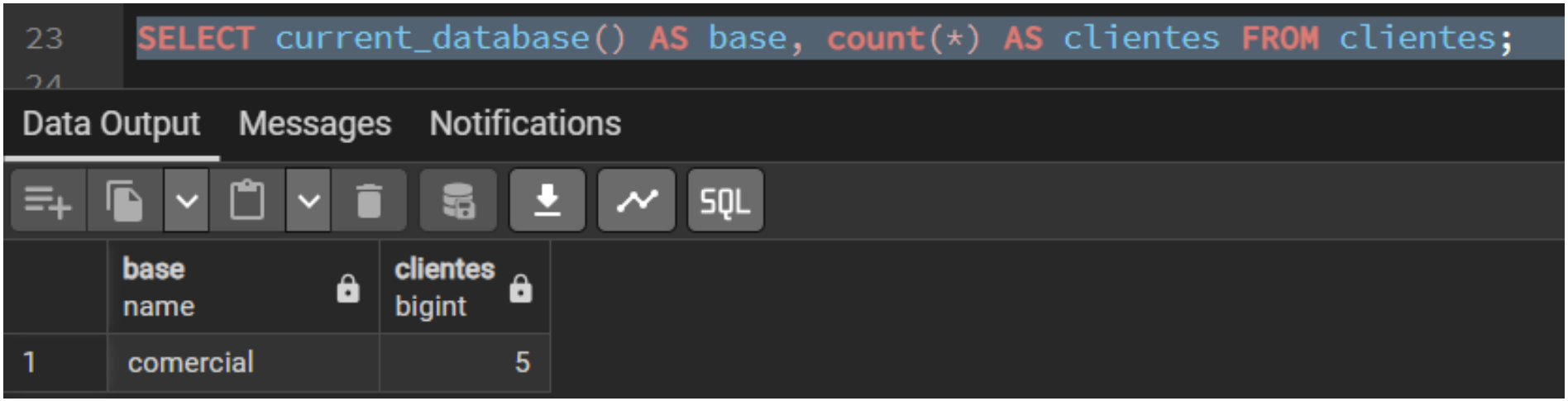
Captura 07. Reglas de autenticación local

```
13  SELECT line_number, type, database, user_name, address, auth_method
14  FROM pg_hba_file_rules
15  WHERE database = '{all}';
16
17
18
```

Data Output | Messages | Notifications

	line_number integer	type text	database text[]	user_name text[]	address text	auth_method text
1	113	local	{all}	{all}	[null]	scram-sha-256
2	115	host	{all}	{all}	127.0.0.1	scram-sha-256
3	117	host	{all}	{all}	::1	scram-sha-256

Captura 10. Cinco clientes en la base comercial



Captura 16. Parámetros de registro antes de la configuración

```
31  SELECT name, setting, source
32  FROM pg_settings
33  WHERE name IN (
34      'log_destination', 'logging_collector', 'log_directory', 'log_filename',
35      'log_connections', 'log_disconnections', 'log_line_prefix',
36      'log_statement', 'log_min_error_statement', 'log_timezone'
37  )
38  ORDER BY name;
39
40
```

Data Output | Messages | Notifications

SQL

	name text	setting text	source text
1	log_connections		default
2	log_destination	stderr	configuration f...
3	log_directory	log	default
4	log_disconnections	off	default
5	log_filename	postgresql-%Y-%m-%d_%H%M%S.l...	default
6	log_line_prefix	%t	configuration f...
7	log_min_error_statem...	error	default
8	log_statement	none	default
9	log_timezone	America/Chihuahua	configuration f...
10	logging_collector	on	configuration f...

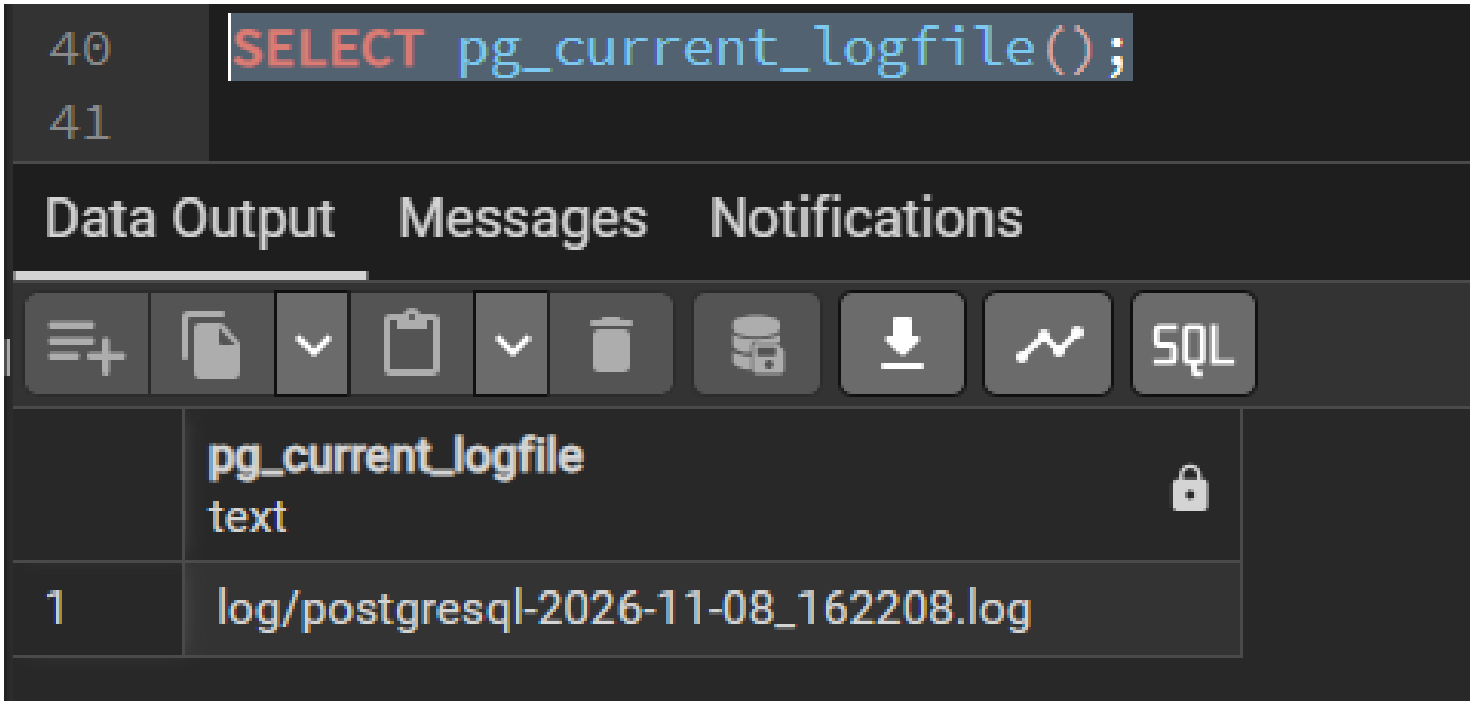
Actividad 2 - Colector y lectura de la bitácora

El colector ya estaba activo. `pg_current_logfile()` devolvió `log/postgresql-2026-11-08_162208.log`. La lectura de los primeros 4096 bytes con `pg_read_file` produjo el error `22021`; el mismo fragmento se pudo leer con `pg_read_binary_file` y `convert_from(..., 'WIN1252')`.

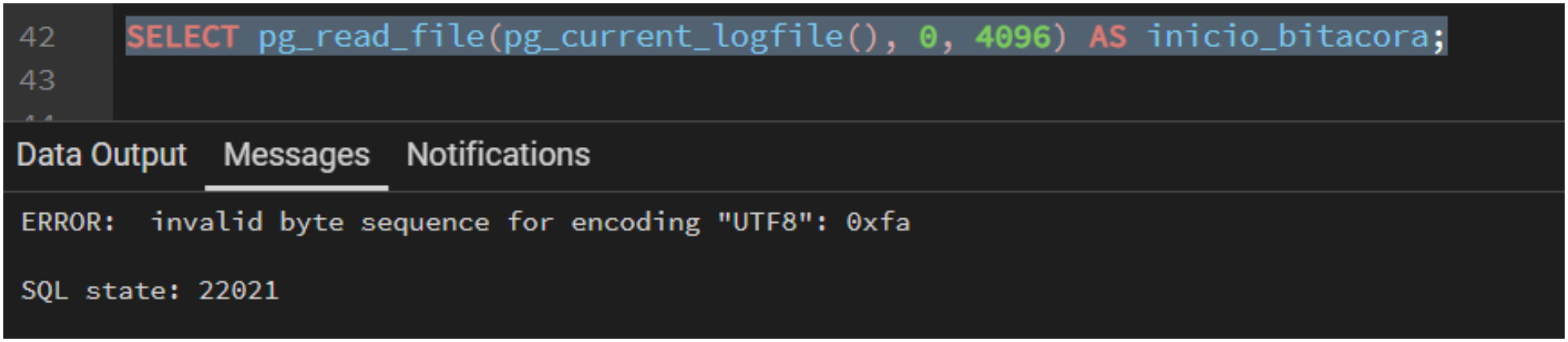
El esquema `dba` y la vista `dba.bitacora` ya existían. PostgreSQL rechazó el intento de crear la vista de nuevo con `42P07`. Los comandos `REVOKE` sobre el esquema y la vista terminaron correctamente. El conteo devolvió **65 líneas no vacías**.

Con `pg_get_viewdef('dba.bitacora'::regclass, true)` se confirmó que la vista obtiene el archivo activo con `pg_current_logfile()`, lo lee con `pg_read_binary_file`, convierte el texto desde `WIN1252` y lo separa por saltos de línea con `regexp_split_to_table`. Se reutilizó la vista existente.

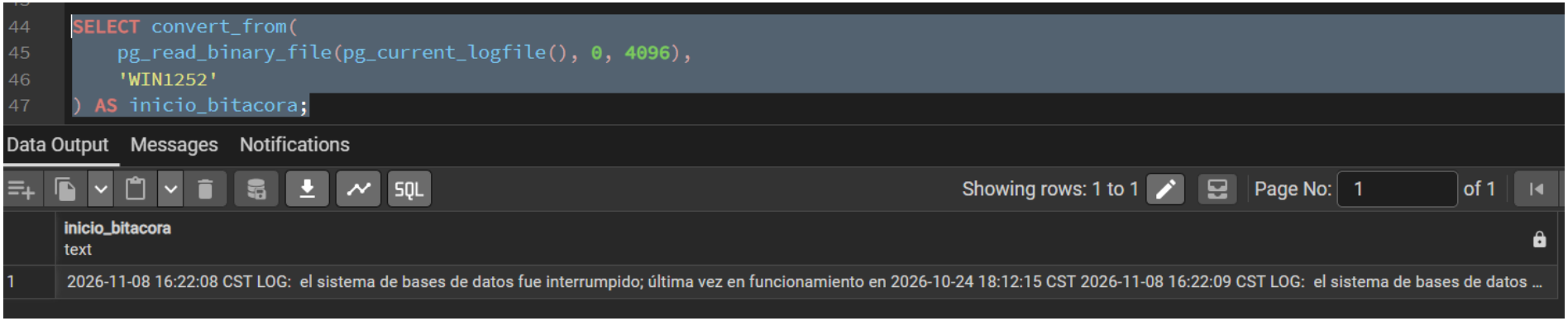
Captura 17. Archivo de bitácora activo



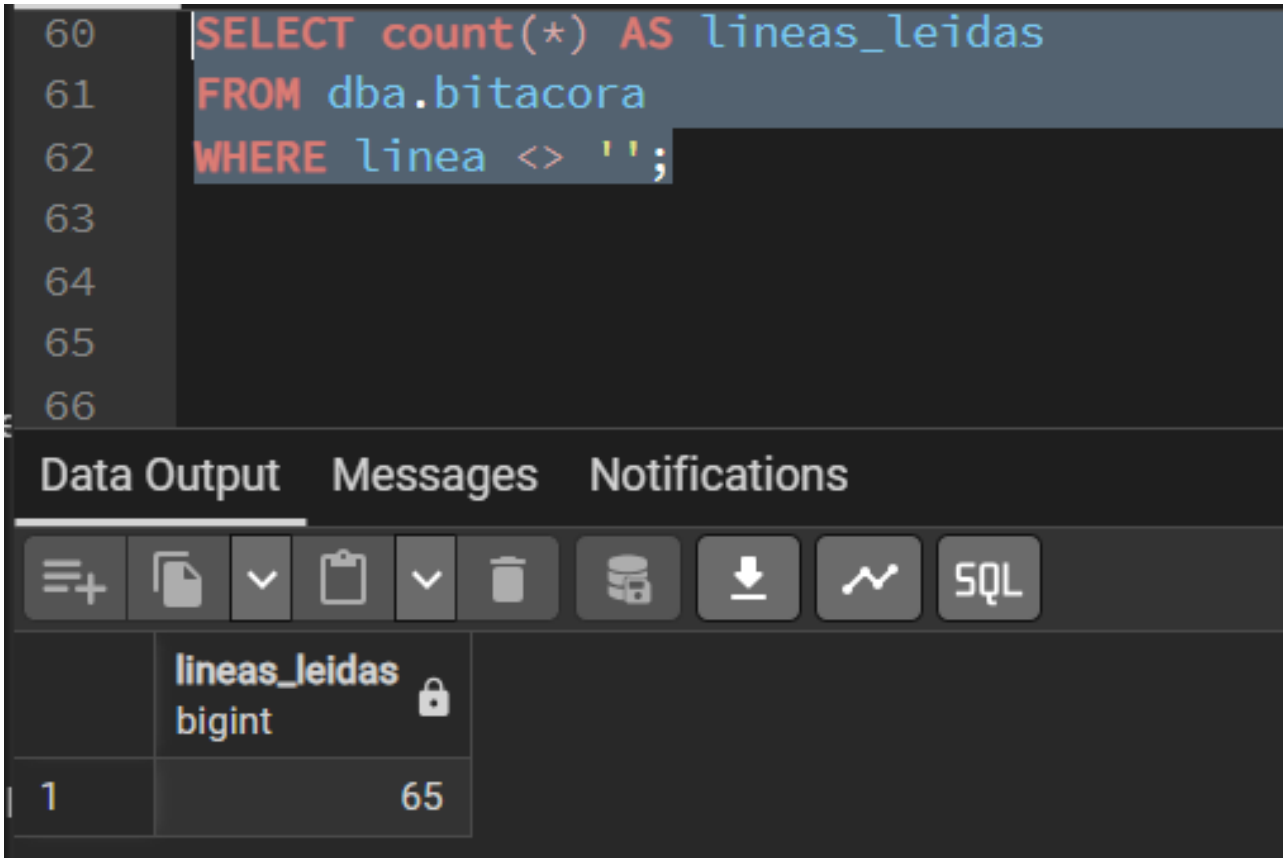
Captura 18. Error de lectura como UTF-8



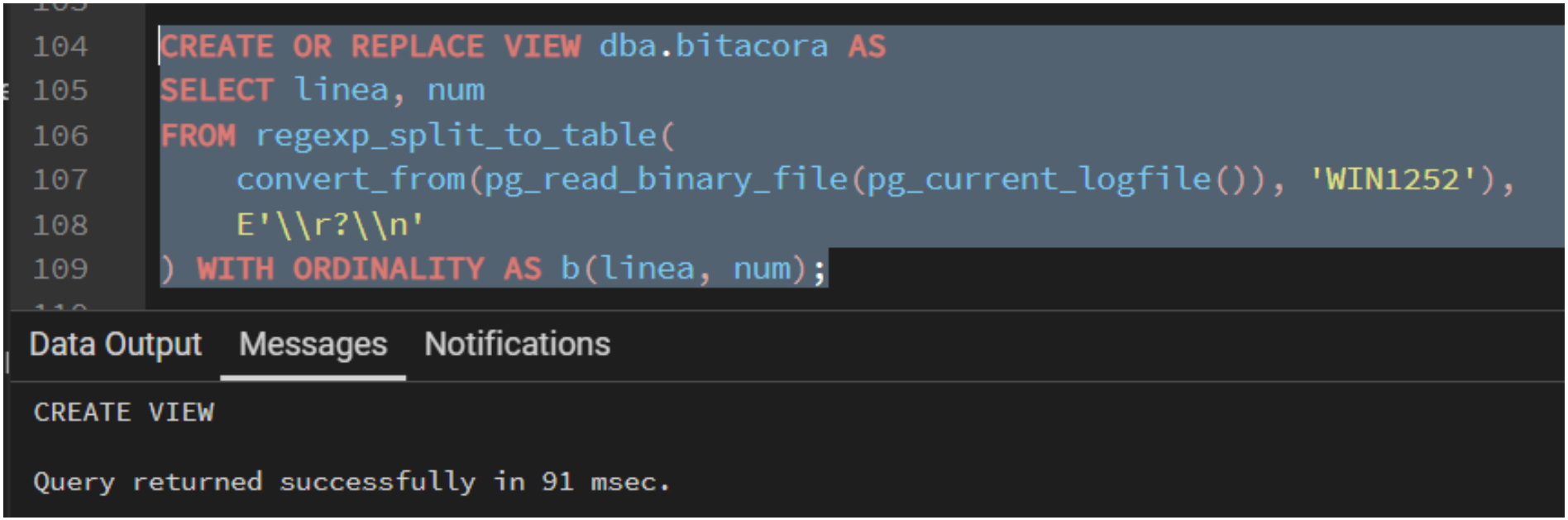
Captura 19. Lectura del archivo mediante conversión WIN1252



Captura 23. Conteo inicial de líneas



Captura 42. Vista con numeración de líneas



Actividad 3 - Incidente sin bitácora

En `comercial`, el RFC ficticio del cliente 3, María García, pasó de `GAMA780912GHI` a `GAMA780912XXX`. La sesión de prueba con `app_ventas` devolvió `UPDATE 1`. La consulta posterior confirmó el valor nuevo y la búsqueda de esa sentencia en `dba.bitacora` devolvió **0 coincidencias**.

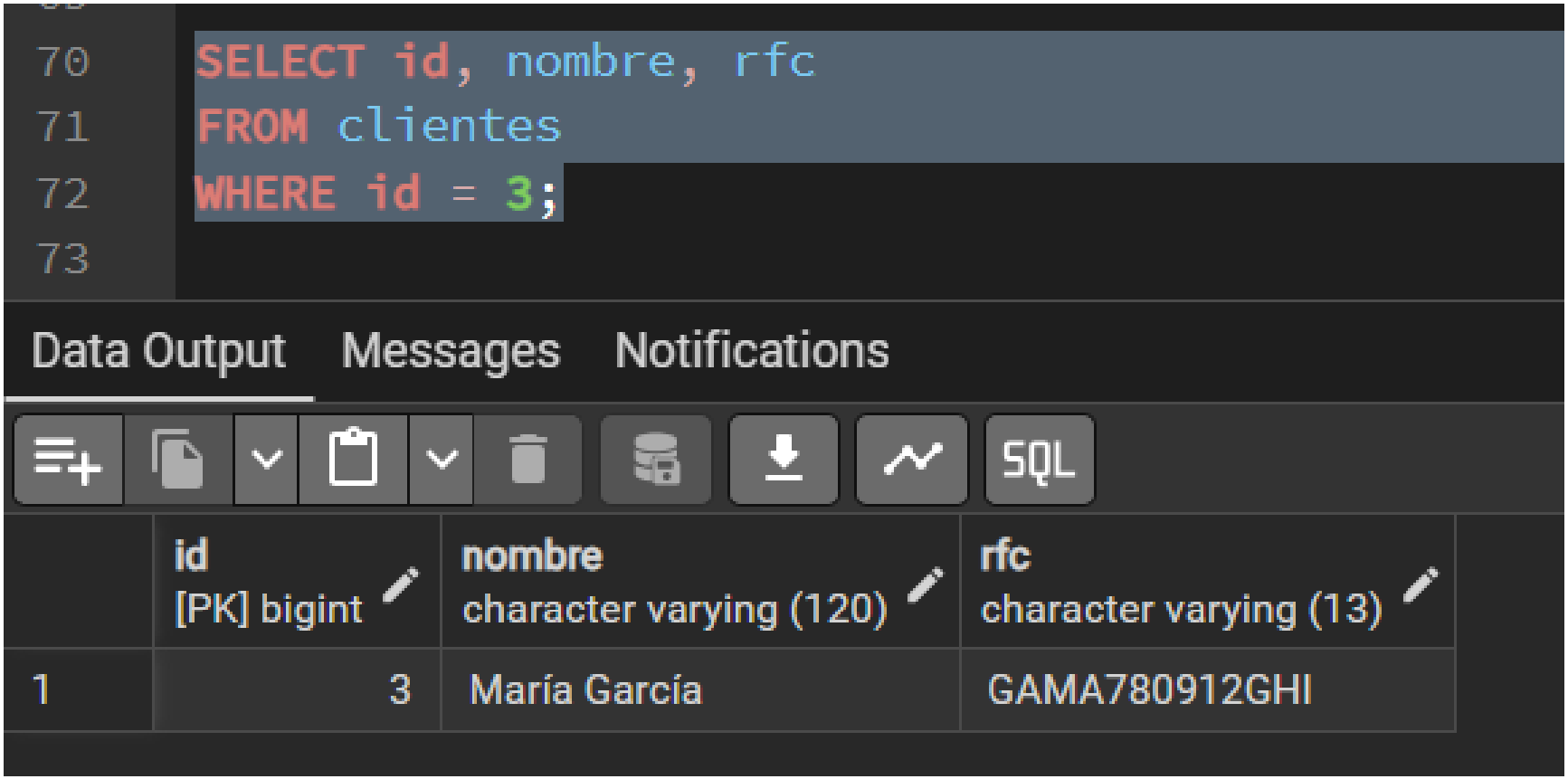
1. La ejecución controlada identifica el rol de prueba, pero el archivo activo no ofrece una entrada que permita atribuirle el cambio a partir de la bitácora sola.

2. La ausencia de entradas no permite descartar a `soporte_ana` por sí sola. El permiso de solo lectura indica que ese rol no podía ejecutar directamente el cambio con los permisos concedidos; eso es evidencia de autorización, no un historial de acciones.

3. El control de acceso permitió el cambio porque `app_ventas` tenía permiso de actualizar. Si actualizar el RFC forma parte de su función, no debía impedirlo; si no, se deberían limitar los permisos por columna.

4. Antes del incidente se necesitaba registrar las sentencias de modificación y el contexto de la sesión. Para conservar también el valor anterior y el nuevo, se necesitaba auditoría de cambios en los datos.

Captura 26. RFC del cliente antes del incidente

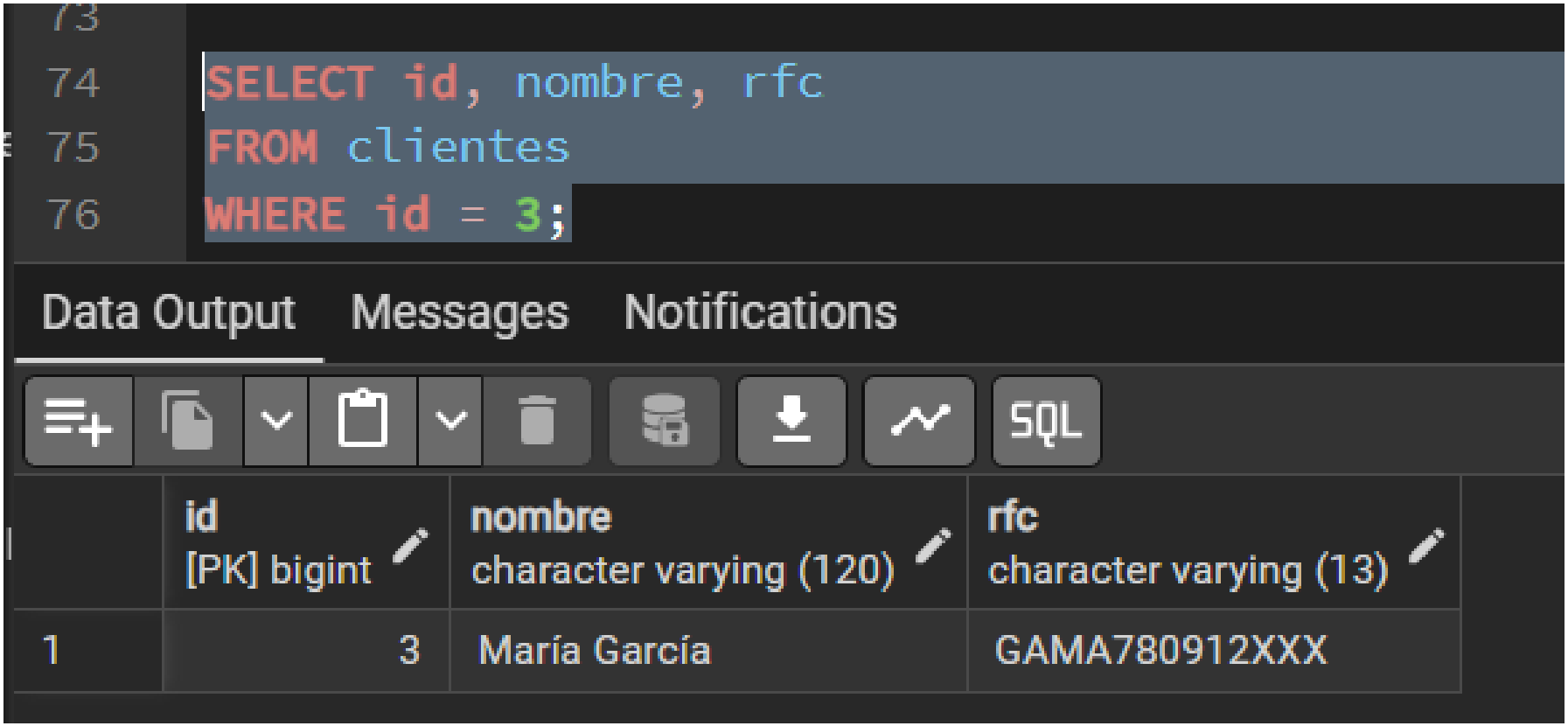


Captura 27. Ejecución controlada del cambio

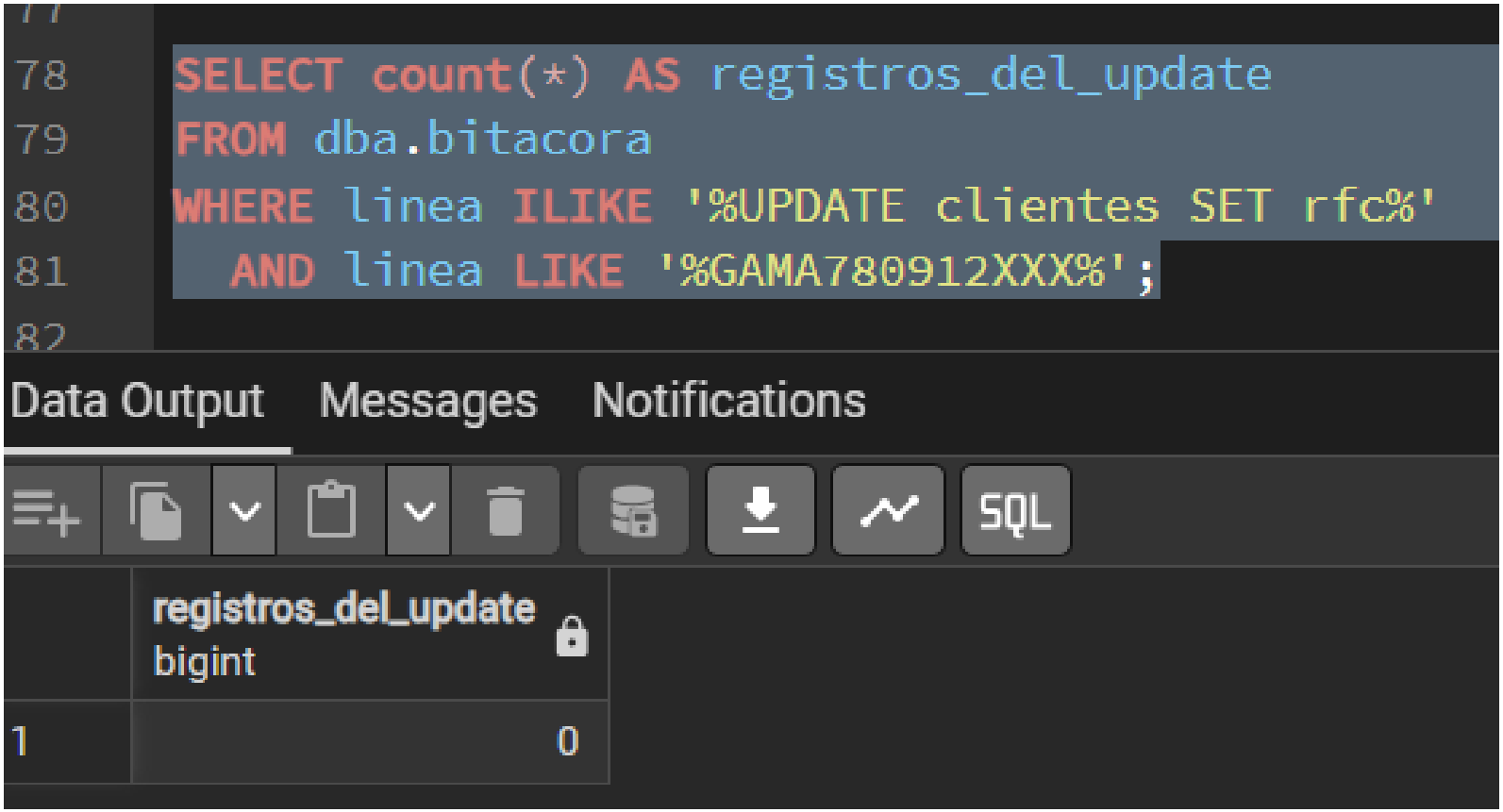
```
PS C:\Users\diego> & "C:\Program Files\PostgreSQL\18\bin\psql.exe" -W "host=127.0.0.1 dbname=comercial user=app_ventas a
pplication_name=app-ventas-web" -c "UPDATE clientes SET rfc = 'GAMA780912XXX' WHERE id = 3;"
Contraseña:

UPDATE 1
```

Captura 28. RFC después del incidente



Captura 29. Búsqueda de la sentencia en la bitácora



Actividad 4 - Configuración de la bitácora

Se ejecutaron cuatro comandos `ALTER SYSTEM` y `pg_reload_conf()` devolvió `true`. La sesión que ya estaba abierta mostró el prefijo nuevo y `log_statement = ddl`, ambos con origen en el archivo de configuración. Los cuatro parámetros mostraron `pending_restart = false`.

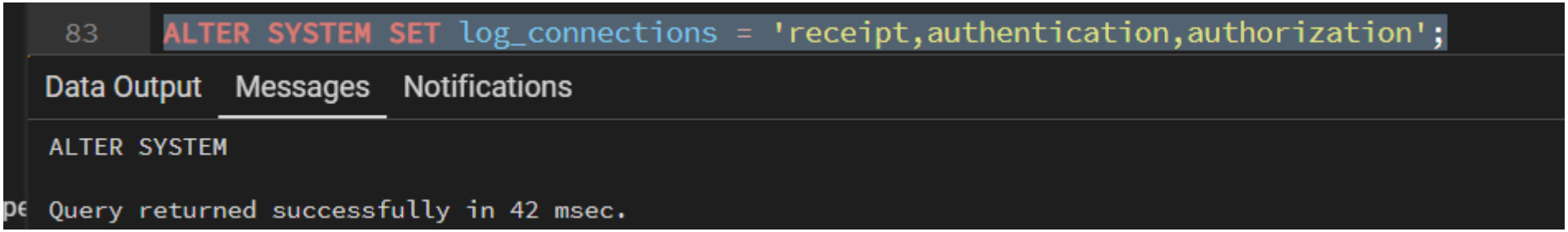
En pgAdmin, `log_connections` aún apareció vacío y `log_disconnections` en `off`, con origen `default`.

Una conexión nueva de `psql` desde PowerShell a `127.0.0.1`, con el usuario `postgres` y la base `comercial`, confirmó `log_connections = receipt,authentication,authorization` y `log_disconnections = on`. Ambos parámetros mostraron `configuration file` como origen.

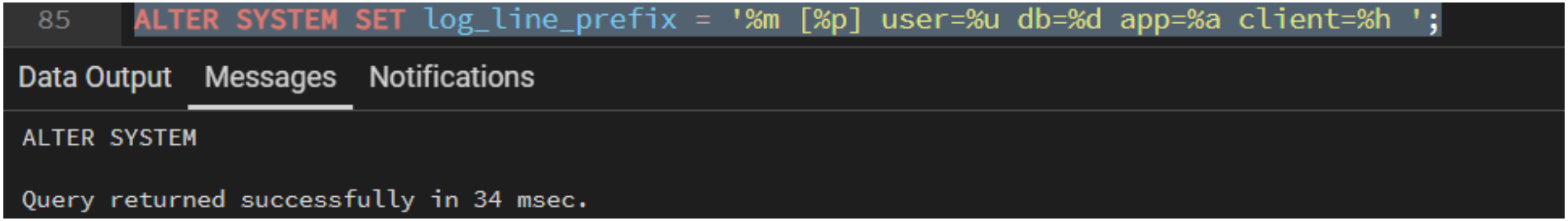
Se abrió una conexión de prueba desde PowerShell como `app_ventas` en `comercial`, con `application_name=lab01-prueba-bitacora`. La consulta devolvió el PID `19628`. Ese PID permite buscar las entradas de conexión y desconexión de la sesión en `dba.bitacora`.

La búsqueda del PID `19628` en `dba.bitacora` devolvió cuatro eventos: recepción desde `127.0.0.1`, autenticación de `app_ventas` mediante SCRAM-SHA-256, autorización en `comercial` y desconexión. La última entrada identifica la aplicación `lab01-prueba-bitacora` y registra una duración de `0:00:00.055`.

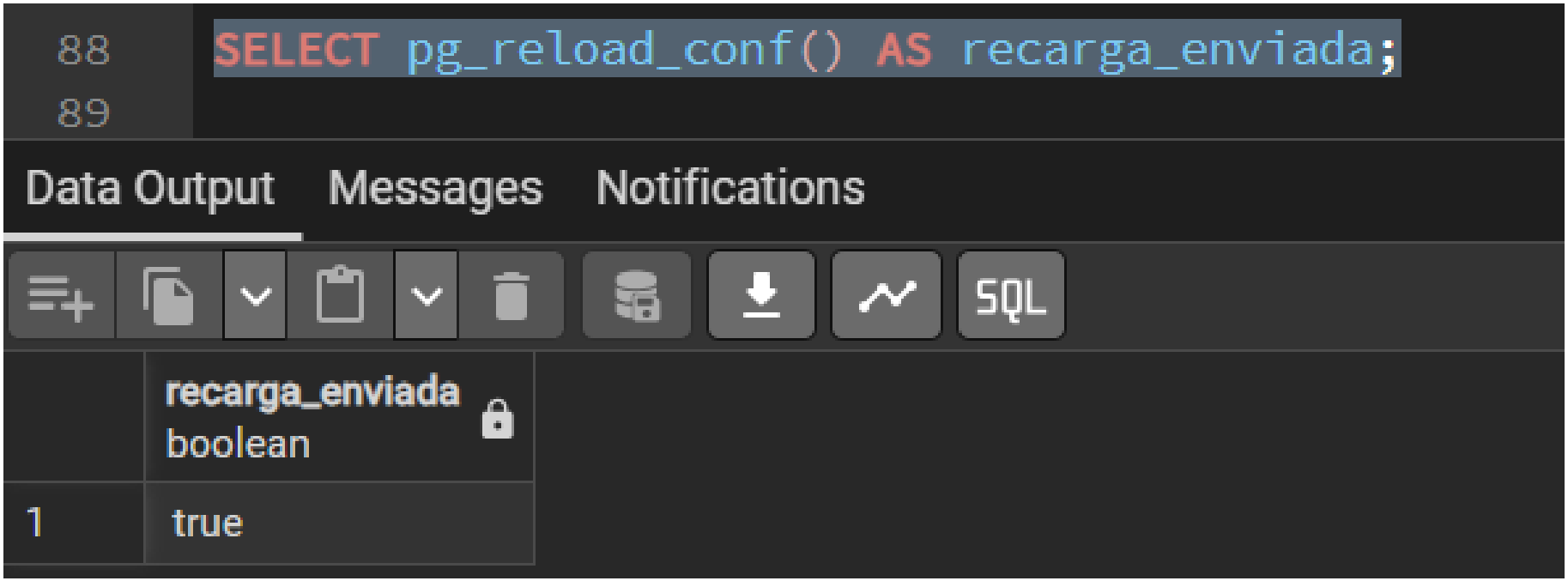
Captura 30. Configuración de eventos de conexión



Captura 32. Configuración del prefijo



Captura 34. Recarga de la configuración



Captura 37. Parámetros efectivos en una conexión nueva

```
PS C:\Users\diego> & "C:\Program Files\PostgreSQL\18\bin\psql.exe" -X -W "host=127.0.0.1 dbname=comercial user=postgres"
 -c "SELECT name, setting, source FROM pg_settings WHERE name IN ('log_connections', 'log_disconnections') ORDER BY name
;"
Contraseña:

        name        |               setting                |       source
--------------------+--------------------------------------+--------------------
 log_connections    | receipt,authentication,authorization | configuration file
 log_disconnections | on                                   | configuration file
(2 filas)
```

Captura 39. Recepción, autenticación, autorización y desconexión

```
100  SELECT linea
101  FROM dba.bitacora
102  WHERE linea LIKE '% [19628] %';
103
```

Data Output | Messages | Notifications

Showing rows: 1 to 4 | Page No: 1 of 1

	linea text
1	2026-11-08 17:12:19.221 CST [19628] user=[desconocido] db=[desconocido] app=[desconocido] client=127.0.0.1 LOG: conexión recibida: host=127.0.0.1 port=52519
2	2026-11-08 17:12:19.240 CST [19628] user=app_ventas db=comercial app=[desconocido] client=127.0.0.1 LOG: conexión autenticada: identidad=«app_ventas» método=scram-sha-256 (C:/Program Files/P…
3	2026-11-08 17:12:19.240 CST [19628] user=app_ventas db=comercial app=[desconocido] client=127.0.0.1 LOG: conexión autorizada: usuario=app_ventas base_de_datos=comercial nombre_de_aplicación…
4	2026-11-08 17:12:19.267 CST [19628] user=app_ventas db=comercial app=lab01-prueba-bitacora client=127.0.0.1 LOG: desconexiÃ³n: duraciÃ³n de sesiÃ³n: 0:00:00.055 usuario=app_ventas base=comerc…

Actividad 5 - Accesos exitosos y fallidos

Desde PowerShell se probaron tres conexiones a `comercial` en `127.0.0.1`. `psql` mostró un error de autenticación para `app_ventas` con una contraseña incorrecta y para `usuario_inexistente`. Con la contraseña correcta y la aplicación `app-ventas-web`, `app_ventas` ejecutó `SELECT count(*) FROM clientes` y obtuvo cinco filas en la tabla.

Se agregó la columna `num` a `dba.bitacora` para ordenar las entradas por su posición en el archivo. La consulta de la guía filtró por `client=127.0.0.1` y devolvió 48 filas. En las últimas filas se identificaron estos eventos del 8 de noviembre de 2026 en la base `comercial`:

Hora (CST)	PID	Usuario	Resultado
17:17:31.243	5316	`app_ventas`	FATAL por contraseña incorrecta. El detalle indica que coincidió una regla de `pg_hba.conf`.
17:17:43.484	21604	`usuario_inexistente`	FATAL de autenticación. La línea `DETALLE` indica que el rol no existe.
17:17:59.943	14396	`app_ventas`	Autenticación mediante SCRAM-SHA-256 y conexión autorizada.
17:17:59.983	14396	`app_ventas`	Desconexión de `app-ventas-web` tras 67 ms.

PowerShell mostró el mismo tipo de error para los dos rechazos. El detalle de la bitácora del servidor permitió identificar el intento con un rol inexistente. El PID `14396` relacionó la autorización con la desconexión de la sesión exitosa.

Preguntas de la guía

1. **¿Qué demuestra el usuario del intento fallido?** Demuestra que un cliente desde `127.0.0.1` presentó el nombre `app_ventas` e intentó autenticarse. Un cliente puede declarar ese usuario sin ser la aplicación de ventas.

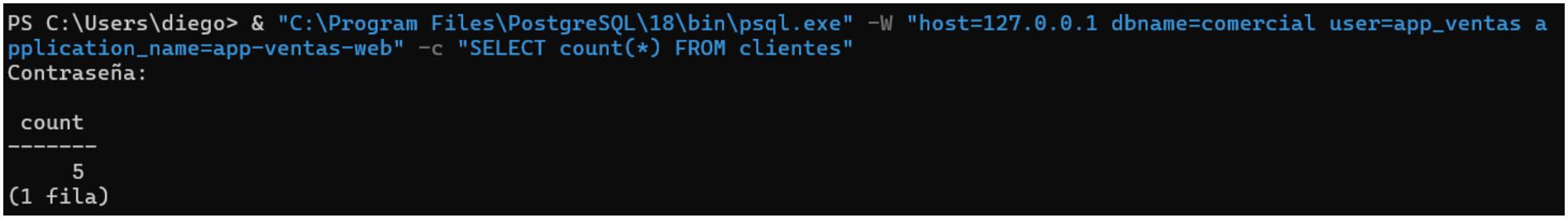
2. **¿Qué detalle queda cuando el rol no existe?** PostgreSQL indica en la bitácora que `usuario_inexistente` no existe. El cliente recibe un error genérico de autenticación, lo que evita revelar qué roles son válidos.

3. **¿Por qué cambia el campo `app`?** En el rechazo, la conexión terminó durante la autenticación y el prefijo aún mostraba la aplicación como desconocida. Al cerrar la sesión exitosa, el prefijo ya incluía el nombre `app-ventas-web` que declaró el cliente.

4. **¿Qué patrón sugiere fuerza bruta?** Muchos fallos en poco tiempo contra el mismo usuario desde una IP, o intentos contra varios usuarios desde el mismo origen. La siguiente consulta propuesta cuenta entradas de autenticación fallida por usuario. Cada entrada representa un fallo registrado por el servidor.

```
SELECT substring(linea FROM 'user=([^ ]+) db=') AS usuario,
count(*) AS intentos_fallidos
FROM dba.bitacora
WHERE linea ~* 'FATAL:.*(password authentication failed|password fall)'
GROUP BY 1
ORDER BY intentos_fallidos DESC, usuario;
```

Captura 41. Acceso autorizado y conteo de clientes



Captura 44. Rechazos y conexión exitosa en el servidor

28	2026-11-08 17:17:05.664 CST [2428] user=app_ventas db=comercial app=[desconocido] client=127.0.0.1 FATAL: la autentificación password falló para el usuario «app_ventas»
29	2026-11-08 17:17:05.664 CST [2428] user=app_ventas db=comercial app=[desconocido] client=127.0.0.1 DETALLE: La conexión coincidió con el archivo «C:/Program Files/PostgreSQL/18/data/pg_hba.con…
30	2026-11-08 17:17:05.791 CST [18304] user=[desconocido] db=[desconocido] app=[desconocido] client=127.0.0.1 LOG: conexión recibida: host=127.0.0.1 port=65379
31	2026-11-08 17:17:05.799 CST [18304] user=app_ventas db=comercial app=[desconocido] client=127.0.0.1 FATAL: la autentificación password falló para el usuario «app_ventas»
32	2026-11-08 17:17:05.799 CST [18304] user=app_ventas db=comercial app=[desconocido] client=127.0.0.1 DETALLE: La conexión coincidió con el archivo «C:/Program Files/PostgreSQL/18/data/pg_hba.co…
33	2026-11-08 17:17:31.225 CST [5316] user=[desconocido] db=[desconocido] app=[desconocido] client=127.0.0.1 LOG: conexión recibida: host=127.0.0.1 port=58754
34	2026-11-08 17:17:31.243 CST [5316] user=app_ventas db=comercial app=[desconocido] client=127.0.0.1 FATAL: la autentificación password falló para el usuario «app_ventas»
35	2026-11-08 17:17:31.243 CST [5316] user=app_ventas db=comercial app=[desconocido] client=127.0.0.1 DETALLE: La conexión coincidió con el archivo «C:/Program Files/PostgreSQL/18/data/pg_hba.con…
36	2026-11-08 17:17:31.374 CST [1496] user=[desconocido] db=[desconocido] app=[desconocido] client=127.0.0.1 LOG: conexión recibida: host=127.0.0.1 port=58756
37	2026-11-08 17:17:31.382 CST [1496] user=app_ventas db=comercial app=[desconocido] client=127.0.0.1 FATAL: la autentificación password falló para el usuario «app_ventas»
38	2026-11-08 17:17:31.382 CST [1496] user=app_ventas db=comercial app=[desconocido] client=127.0.0.1 DETALLE: La conexión coincidió con el archivo «C:/Program Files/PostgreSQL/18/data/pg_hba.con…
39	2026-11-08 17:17:43.466 CST [21604] user=[desconocido] db=[desconocido] app=[desconocido] client=127.0.0.1 LOG: conexión recibida: host=127.0.0.1 port=58757
40	2026-11-08 17:17:43.484 CST [21604] user=usuario_inexistente db=comercial app=[desconocido] client=127.0.0.1 FATAL: la autentificación password falló para el usuario «usuario_inexistente»
41	2026-11-08 17:17:43.484 CST [21604] user=usuario_inexistente db=comercial app=[desconocido] client=127.0.0.1 DETALLE: No existe el rol «usuario_inexistente».
42	2026-11-08 17:17:43.506 CST [1660] user=[desconocido] db=[desconocido] app=[desconocido] client=127.0.0.1 LOG: conexión recibida: host=127.0.0.1 port=58758
43	2026-11-08 17:17:43.514 CST [1660] user=usuario_inexistente db=comercial app=[desconocido] client=127.0.0.1 FATAL: la autentificación password falló para el usuario «usuario_inexistente»
44	2026-11-08 17:17:43.514 CST [1660] user=usuario_inexistente db=comercial app=[desconocido] client=127.0.0.1 DETALLE: No existe el rol «usuario_inexistente».
45	2026-11-08 17:17:59.925 CST [14396] user=[desconocido] db=[desconocido] app=[desconocido] client=127.0.0.1 LOG: conexión recibida: host=127.0.0.1 port=58759
46	2026-11-08 17:17:59.943 CST [14396] user=app_ventas db=comercial app=[desconocido] client=127.0.0.1 LOG: conexión autenticada: identidad=«app_ventas» método=scram-sha-256 (C:/Program Files/Po…
47	2026-11-08 17:17:59.943 CST [14396] user=app_ventas db=comercial app=[desconocido] client=127.0.0.1 LOG: conexión autorizada: usuario=app_ventas base_de_datos=comercial nombre_de_aplicación=…
48	2026-11-08 17:17:59.983 CST [14396] user=app_ventas db=comercial app=app-ventas-web client=127.0.0.1 LOG: desconexiÃ³n: duraciÃ³n de sesiÃ³n: 0:00:00.067 usuario=app_ventas base=comercial host…

Actividad 6 - Cambios de privilegios y de estructura

Se ejecutó el bloque de `REVOKE`, `GRANT` y `ALTER TABLE` en pgAdmin. La pestaña Messages mostró `ALTER TABLE` y confirmó que el bloque terminó correctamente. A continuación, se ejecutó el bloque de creación y eliminación de `rol_temporal`. La pestaña Messages mostró `DROP ROLE`.

La consulta de la guía filtró por `statement:` y mostró los encabezados de los bloques. La consulta adicional recuperó también sus líneas contiguas y ocultó la contraseña en la salida. Los bloques quedaron registrados bajo el PID `12184`, con `postgres` en `comercial` desde pgAdmin y el cliente local `::1`.

Hora (CST)	Líneas de bitácora	Sentencias
17:22:44.710	128-130	`REVOKE UPDATE`, `GRANT UPDATE` y `ALTER TABLE` para agregar `segmento`.
17:23:57.965	133-134	`CREATE ROLE rol_temporal` y `DROP ROLE rol_temporal`.
17:25:31.520	135-136	Segunda ejecución del bloque de creación y eliminación del rol temporal.

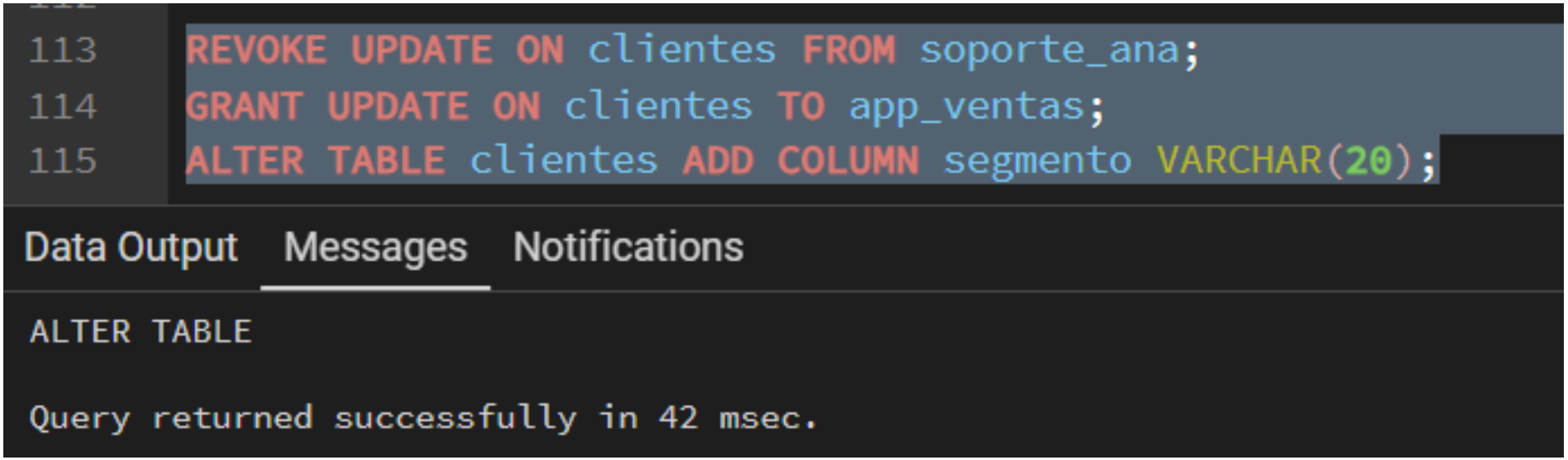
Preguntas de la guía

1. **¿Qué tipo de evento se registró?** `REVOKE` y `GRANT` son cambios de privilegios. `ALTER TABLE` es un cambio de estructura porque agregó una columna a `clientes`.

2. **¿Por qué se registran `GRANT` y `REVOKE` con `log_statement = 'ddl'`?** PostgreSQL incluye en ese nivel las sentencias que administran privilegios, además de las que definen o modifican objetos.

3. **¿Quién debería poder leer la bitácora si contiene contraseñas?** Solo el personal autorizado según sus funciones de administración, auditoría o seguridad. La prueba mostró que el texto de la contraseña quedó en el registro tras eliminar el rol. La consulta solo lo ocultó en su resultado. La evidencia compartida debe usar la salida redactada.

Captura 46. Cambios de privilegios y estructura



Captura 50. Creación y baja del rol temporal con la contraseña oculta

3	130	ALTER TABLE clientes ADD COLUMN segmento VARCHAR(20);
4	133	2026-11-08 17:23:57.965 CST [12184] user=postgres db=comercial app=pgAdmin 4 - CONN:7802833 client=::1 LOG: statement: CREATE ROLE rol_temporal LOGIN PASSWORD '<REDAC…
5	134	DROP ROLE rol_temporal;
6	135	2026-11-08 17:25:31.520 CST [12184] user=postgres db=comercial app=pgAdmin 4 - CONN:7802833 client=::1 LOG: statement: CREATE ROLE rol_temporal LOGIN PASSWORD '<REDAC…
7	136	DROP ROLE rol_temporal;

Actividad 7 - Modificaciones de datos con `ddl` y `mod`

Con `log_statement = 'ddl'`, `app_ventas` actualizó el RFC del cliente con `id = 3` a `GAMA780912XYZ` desde `app-ventas-web`. PowerShell devolvió `UPDATE 1`. La consulta de las tres entradas más recientes de `app_ventas` mostró la autenticación, la autorización y la desconexión del PID `6340` desde `127.0.0.1`. La sentencia `UPDATE` no apareció en ese resultado.

Como `postgres`, se ejecutó `ALTER SYSTEM SET log_statement = 'mod'` y `pg_reload_conf()` devolvió `true`. En una conexión nueva, `app_ventas` actualizó el teléfono del cliente `id = 3` a `6679990000` y obtuvo `UPDATE 1`. En la misma sesión, el `SELECT` devolvió una fila con el nombre de María García y el RFC `GAMA780912XYZ`.

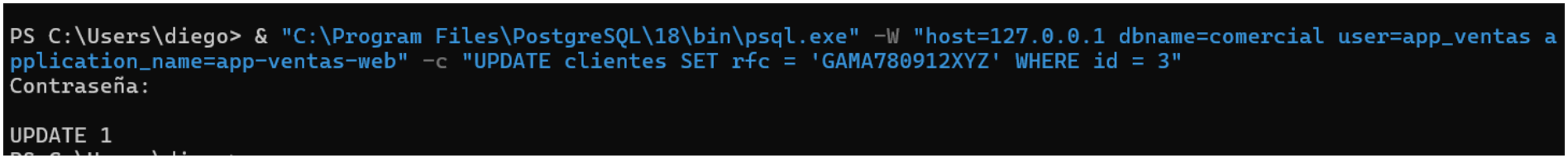
La consulta de la guía devolvió tres filas relacionadas con `ALTER SYSTEM`. Después, `SHOW log_statement` confirmó el valor efectivo `mod`. La búsqueda por el texto del `UPDATE` encontró la línea 177 del 8 de noviembre de 2026 a las 17:31:24.803 CST, con el PID `4812`, el rol `app_ventas`, la aplicación `app-ventas-web` y el cliente `127.0.0.1`. El registro usó el prefijo `LOG: sentencia:`, de modo que el filtro original por `statement:` omitió esa línea.

Preguntas de la guía

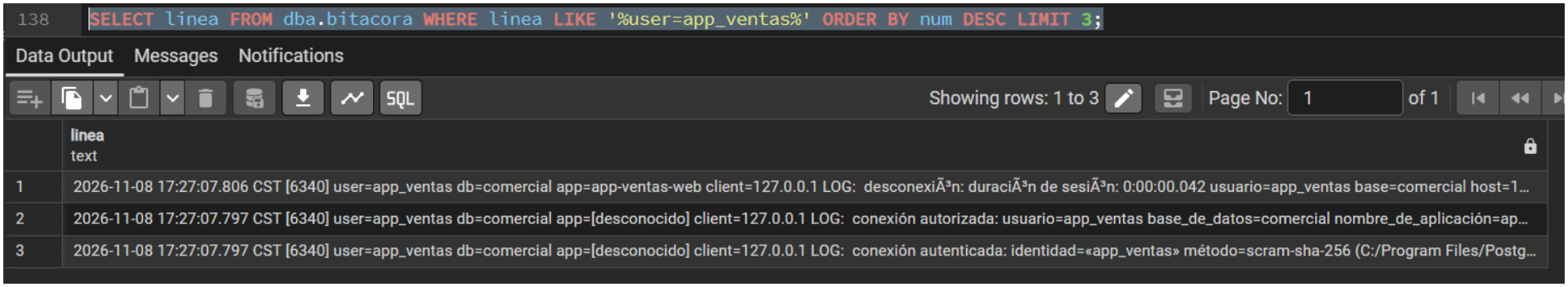
1. **¿Qué nivel habría registrado la lectura del RFC y qué costo tendría?** El nivel `all` registra las sentencias `SELECT`. En un servidor con miles de consultas por minuto aumentaría el volumen de la bitácora, la escritura a disco y el espacio necesario para conservar los registros.

2. **¿Qué falta para revertir el cambio con certeza?** Falta el valor anterior del teléfono. La sentencia registrada contiene el nuevo número y el identificador del cliente. Para recuperar el valor previo se necesita otra fuente de evidencia, como la auditoría por fila de la Parte B.

Captura 51. Actualización del RFC con nivel ddl



Captura 52. Entradas de sesión registradas con ddl

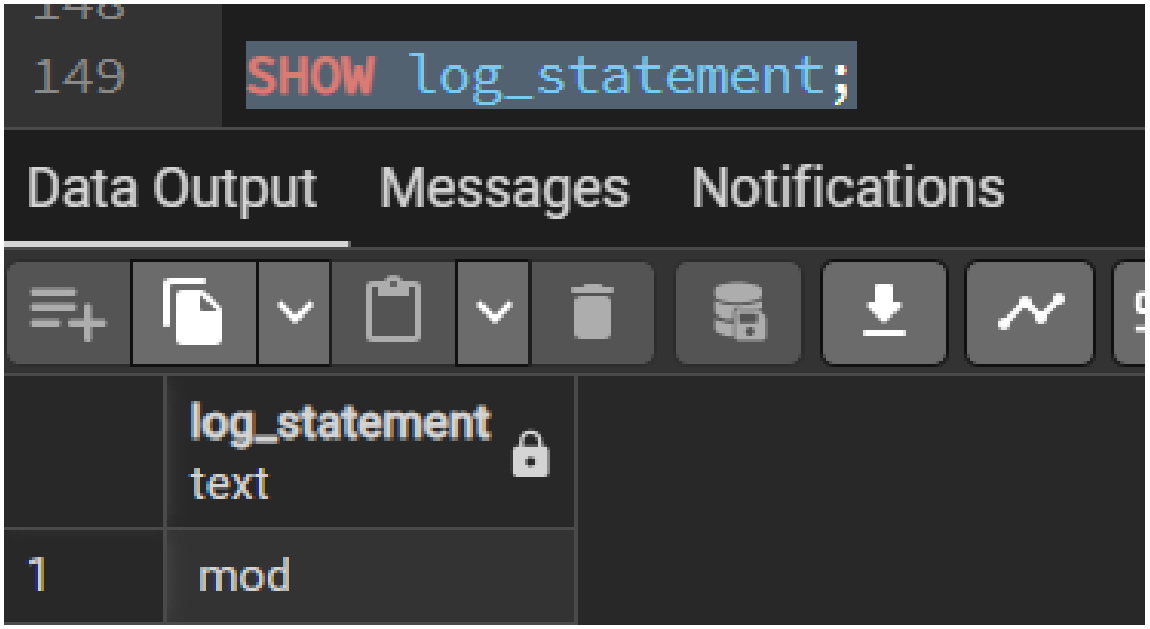


Captura 55. Actualización del teléfono y consulta de lectura

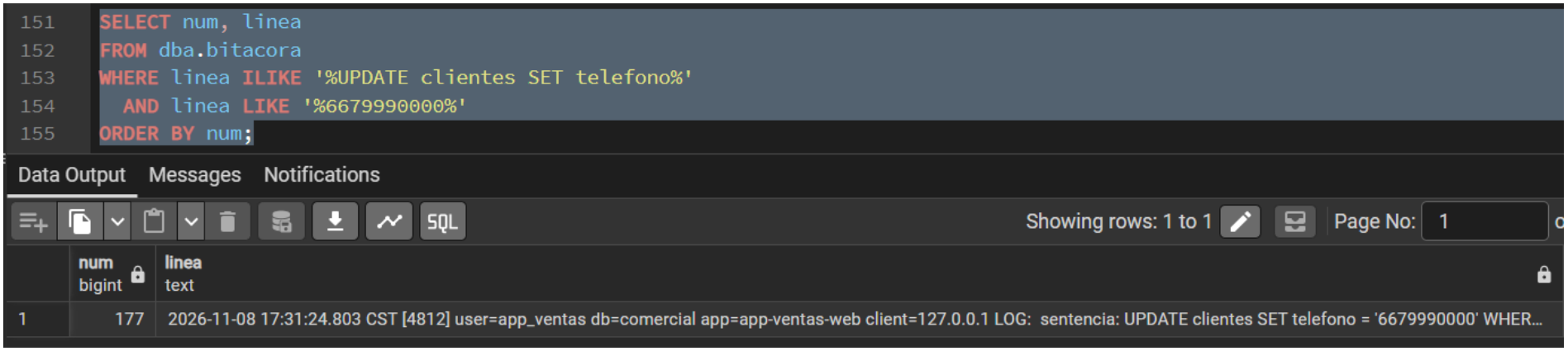
```
PS C:\Users\diego> & "C:\Program Files\PostgreSQL\18\bin\psql.exe" -W "host=127.0.0.1 dbname=comercial user=app_ventas a
pplication_name=app-ventas-web" -c "UPDATE clientes SET telefono = '6679990000' WHERE id = 3" -c "SELECT nombre, rfc FRO
M clientes WHERE id = 3"
Contraseña:

UPDATE 1
    nombre    |      rfc
--------------+---------------
 MarÝa GarcÝa | GAMA780912XYZ
(1 fila)
```

Captura 57. Nivel mod activo



Captura 58. Sentencia de actualización del teléfono



Actividad 8 - Errores y operaciones rechazadas

Desde PowerShell, `soporte_ana` se conectó a `comercial` por `127.0.0.1` e intentó ejecutar `DELETE FROM clientes WHERE id = 5`. PostgreSQL rechazó la sentencia con un error de permiso denegado sobre la tabla `clientes`.

En pgAdmin se intentó ejecutar `DROP ROLE soporte_ana` dentro de una transacción de prueba. PostgreSQL devolvió el código `2BP01` e identificó los privilegios sobre `clientes` como la dependencia que impidió eliminar el rol.

El filtro de la guía devolvió cero filas. El filtro adaptado aceptó etiquetas en español e inglés y uno o más espacios después de cada etiqueta. Devolvió 18 filas, entre ellas los errores, los detalles y las sentencias de ambos casos.

Hora (CST)	PID	Rol y origen	Resultado
17:35:09.556	18388	`soporte_ana`, `psql`, `127.0.0.1`	`DELETE` rechazado por falta de permiso sobre `clientes`.
17:37:05.016	12184	`postgres`, pgAdmin, `::1`	`DROP ROLE` rechazado por los privilegios de `soporte_ana` sobre `clientes`.

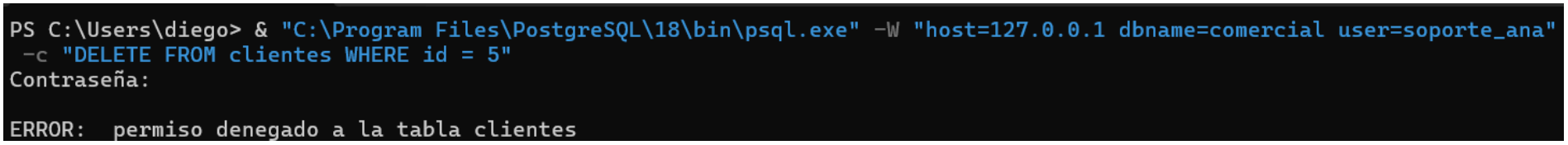
Preguntas de la guía

1. **¿Qué efecto tuvo el intento de `soporte_ana` y por qué se audita?** El control de permisos impidió que se borrara la fila. Registrar el intento permite investigar el uso de una cuenta fuera de sus funciones y detectar si se repite.

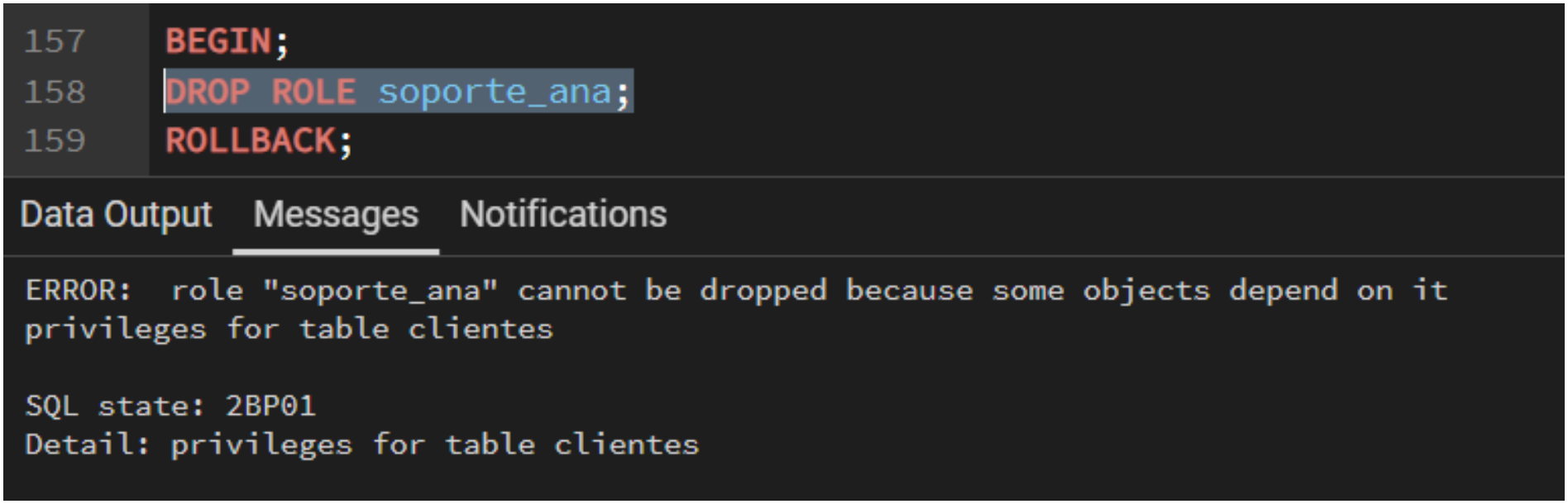
2. **¿En qué se distinguen la autenticación y la autorización?** La autenticación comprueba las credenciales al conectarse. La autorización verifica si el rol ya autenticado tiene permiso para ejecutar una operación.

3. **¿En qué orden se da de baja un rol?** Primero se revisan sus dependencias, se revocan sus privilegios y se reasignan sus objetos si los tiene. Después se elimina el rol. En esta prueba el impedimento fueron los privilegios sobre `clientes`.

Captura 59. Borrado rechazado



Captura 60. Eliminación de rol rechazada



Captura 62. Errores y sentencias en la bitácora

Data Output | Messages | Notifications

Showing rows: 1 to 18 | Page No: 1 of 1

	linea text
1	2026-11-08 17:35:09.550 CST [18388] user=soporte_ana db=comercial app=psql client=127.0.0.1 LOG: sentencia: DELETE FROM clientes WHERE id = 5
2	2026-11-08 17:35:09.556 CST [18388] user=soporte_ana db=comercial app=psql client=127.0.0.1 ERROR: permiso denegado a la tabla clientes
3	2026-11-08 17:35:09.556 CST [18388] user=soporte_ana db=comercial app=psql client=127.0.0.1 SENTENCIA: DELETE FROM clientes WHERE id = 5
4	2026-11-08 17:35:30.348 CST [21808] user=soporte_ana db=comercial app=psql client=127.0.0.1 LOG: sentencia: DELETE FROM clientes WHERE id = 5
5	2026-11-08 17:35:30.350 CST [21808] user=soporte_ana db=comercial app=psql client=127.0.0.1 ERROR: permiso denegado a la tabla clientes
6	2026-11-08 17:35:30.350 CST [21808] user=soporte_ana db=comercial app=psql client=127.0.0.1 SENTENCIA: DELETE FROM clientes WHERE id = 5
7	2026-11-08 17:36:30.946 CST [12184] user=postgres db=comercial app=pgAdmin 4 - CONN:7802833 client=::1 ERROR: role "soporte_ana" cannot be dropped because some objects depe…
8	2026-11-08 17:36:30.946 CST [12184] user=postgres db=comercial app=pgAdmin 4 - CONN:7802833 client=::1 DETAIL: privileges for table clientes
9	2026-11-08 17:36:38.106 CST [12184] user=postgres db=comercial app=pgAdmin 4 - CONN:7802833 client=::1 LOG: statement: DROP ROLE soporte_ana;
10	2026-11-08 17:36:38.106 CST [12184] user=postgres db=comercial app=pgAdmin 4 - CONN:7802833 client=::1 SENTENCIA: DROP ROLE soporte_ana;
11	2026-11-08 17:37:05.016 CST [12184] user=postgres db=comercial app=pgAdmin 4 - CONN:7802833 client=::1 LOG: statement: DROP ROLE soporte_ana;
12	2026-11-08 17:37:05.016 CST [12184] user=postgres db=comercial app=pgAdmin 4 - CONN:7802833 client=::1 ERROR: role "soporte_ana" cannot be dropped because some objects depe…
13	2026-11-08 17:37:05.016 CST [12184] user=postgres db=comercial app=pgAdmin 4 - CONN:7802833 client=::1 DETAIL: privileges for table clientes
14	2026-11-08 17:37:05.016 CST [12184] user=postgres db=comercial app=pgAdmin 4 - CONN:7802833 client=::1 SENTENCIA: DROP ROLE soporte_ana;
15	2026-11-08 17:37:20.525 CST [12184] user=postgres db=comercial app=pgAdmin 4 - CONN:7802833 client=::1 LOG: statement: DROP ROLE soporte_ana;
16	2026-11-08 17:37:20.525 CST [12184] user=postgres db=comercial app=pgAdmin 4 - CONN:7802833 client=::1 ERROR: role "soporte_ana" cannot be dropped because some objects depe…
17	2026-11-08 17:37:20.525 CST [12184] user=postgres db=comercial app=pgAdmin 4 - CONN:7802833 client=::1 DETAIL: privileges for table clientes
18	2026-11-08 17:37:20.525 CST [12184] user=postgres db=comercial app=pgAdmin 4 - CONN:7802833 client=::1 SENTENCIA: DROP ROLE soporte_ana;

Actividad 9 - Sesiones activas con `pg_stat_activity`

La consulta de sesiones de cliente mostró dos conexiones activas de `reporte-mensual` desde `127.0.0.1`, con los PID `19748` y `5348`. Ambas estaban ejecutando `SELECT pg_sleep(90)`. Se eligió el PID `19748`, que corresponde a la conexión iniciada más recientemente, para cotejar la sesión con la bitácora.

El servidor recibió la conexión del PID `19748` desde `127.0.0.1` el 8 de noviembre de 2026 a las 17:40:55.752 CST. Registró la autenticación de `soporte_ana` mediante SCRAM-SHA-256 y la autorización en `comercial` a las 17:40:55.770 CST. Después de que PowerShell devolvió el resultado, la consulta de `dba.bitacora` mostró también la desconexión de `reporte-mensual` a las 17:42:25.817 CST. El servidor registró una duración de 90,074 segundos.

Preguntas de la guía

1. **¿Cómo se relaciona la sesión con la bitácora?** El campo `pid` de `pg_stat_activity` coincide con el identificador del proceso en el prefijo de la bitácora. En esta prueba fue `19748`.

2. **¿Qué conserva la evidencia cuando termina el reporte?** La bitácora conserva las entradas de conexión y desconexión. `pg_stat_activity` muestra las sesiones presentes en el momento de la consulta.

3. **¿Por qué el ejemplo del administrador no muestra `client_addr`?** Una conexión por socket Unix carece de dirección IP y devuelve `NULL`. En la práctica, pgAdmin usó TCP local y el administrador apareció con `::1`.

Captura 63. Reporte en ejecución

```
169  SELECT pid, usename, application_name, client_addr, backend_start, state, left(query, 30) AS query
170  FROM pg_stat_activity
171  WHERE backend_type = 'client backend';
```

Data Output | Messages | Notifications

Showing rows: 1 to 7 | Page No:

	pid integer	usename name	application_name text	client_addr inet	backend_start timestamp with time zone	state text	query text
1	12184	postgres	pgAdmin 4 - CONN:7802833	::1	2026-11-08 16:37:27.418155-...	active	SELECT pid, usename, applic...
2	21488	postgres	pgAdmin 4 - DB:comercial	::1	2026-11-08 16:37:13.425446-...	idle	
3	15428	postgres	pgAdmin 4 - CONN:1209507	::1	2026-11-08 16:27:42.352899-...	idle	SELECT oid, pg_catalog.form...
4	3112	postgres	pgAdmin 4 - DB:db_20558848_diego_villarr...	::1	2026-11-08 16:27:38.019397-...	idle	SELECT
5	6024	postgres	pgAdmin 4 - DB:postgres	::1	2026-11-08 16:27:37.666025-...	idle	SELECT name, vartype, min_v...
6	19748	soporte_a...	reporte-mensual	127.0.0.1	2026-11-08 17:40:55.742755-...	active	SELECT pg_sleep(90)
7	5348	soporte_a...	reporte-mensual	127.0.0.1	2026-11-08 17:40:02.110943-...	active	SELECT pg_sleep(90)

Captura 66. Desconexión del mismo proceso

```
178  SELECT linea
179  FROM dba.bitacora
180  WHERE linea LIKE '% [19748] %'
181  ORDER BY num;
```

Data Output | Messages | Notifications

Showing rows: 1 to 4 | Page No: 1 of 1

	linea text
1	2026-11-08 17:40:55.752 CST [19748] user=[desconocido] db=[desconocido] app=[desconocido] client=127.0.0.1 LOG: conexión recibida: host=127.0.0.1 port=60698
2	2026-11-08 17:40:55.770 CST [19748] user=soporte_ana db=comercial app=[desconocido] client=127.0.0.1 LOG: conexión autenticada: identidad=«soporte_ana» método=scram-sha-256 (C:/Program Files/…
3	2026-11-08 17:40:55.770 CST [19748] user=soporte_ana db=comercial app=[desconocido] client=127.0.0.1 LOG: conexión autorizada: usuario=soporte_ana base_de_datos=comercial nombre_de_aplicación…
4	2026-11-08 17:42:25.817 CST [19748] user=soporte_ana db=comercial app=reporte-mensual client=127.0.0.1 LOG: desconexiÃ³n: duraciÃ³n de sesiÃ³n: 0:01:30.074 usuario=soporte_ana base=comercial ho…

Actividad 10 - Diseño del registro de auditoría

En `comercial`, se creó el esquema `auditoria`, se revocó el acceso del rol `PUBLIC` al esquema y se creó `auditoria.cambios_clientes`. PgAdmin confirmó el resultado con `CREATE TABLE`.

Campo	Tipo	Qué permite saber	¿Puede ser nulo?
`id_evento`	BIGSERIAL	Qué entrada identifica el cambio.	No, es la clave primaria.
`momento`	TIMESTAMPTZ	Cuándo ocurrió el cambio.	No, tiene `NOT NULL` y el valor por defecto `now()`.
`usuario_sesion`	TEXT	Qué rol inició sesión.	No, se toma de `session_user`.
`usuario_efectivo`	TEXT	Con qué rol efectivo se insertó la evidencia.	No, se toma de `current_user`.
`aplicacion`	TEXT	Qué nombre de aplicación declaró el cliente.	El esquema admite `NULL`. Si la sesión no declara un nombre, el valor por defecto puede ser una cadena vacía.
`ip_cliente`	INET	Desde qué dirección llegó la conexión.	Sí, una conexión por socket Unix no tiene IP.
`operacion`	TEXT	Si el cambio fue un alta, una modificación o una baja.	No, la operación es obligatoria.
`id_cliente`	BIGINT	A qué cliente corresponde el cambio.	El esquema admite `NULL`. El disparador de la guía toma la clave primaria de la fila afectada.
`valores_antes`	JSONB	Cómo era el registro antes de la operación.	Sí, un alta no tiene fila previa.
`valores_despues`	JSONB	Cómo quedó el registro después de la operación.	Sí, una baja no deja una fila posterior.

Preguntas de la guía

1. **¿Por qué `TIMESTAMPTZ`?** Identifica un instante que se puede comparar con eventos registrados en otras zonas horarias. PostgreSQL lo presenta en la zona configurada para la sesión.

2. **¿Por qué guardar la fila completa en `JSONB`?** Permite comparar el estado anterior y posterior, detectar qué campos cambiaron y conservar los campos presentes en ese momento aunque la tabla se modifique después.

3. **¿Qué implica guardar el RFC?** La auditoría contiene datos personales y debe protegerse como tal. Su acceso se limita a los roles autorizados para investigar cambios y sus copias requieren el mismo control.

Captura 67. Creación de la tabla de auditoría

```
183  CREATE SCHEMA auditoria;
184  REVOKE ALL ON SCHEMA auditoria FROM PUBLIC;
185  CREATE TABLE auditoria.cambios_clientes (
186      id_evento BIGSERIAL PRIMARY KEY,
187      momento TIMESTAMPTZ NOT NULL DEFAULT now(),
188      usuario_sesion TEXT NOT NULL DEFAULT session_user,
189      usuario_efectivo TEXT NOT NULL DEFAULT current_user,
190      aplicacion TEXT DEFAULT current_setting('application_name'),
191      ip_cliente INET DEFAULT inet_client_addr(),
192      operacion TEXT NOT NULL,
193      id_cliente BIGINT,
194      valores_antes JSONB,
195      valores_despues JSONB
196  );
```

Data Output | Messages | Notifications

```
CREATE TABLE

Query returned successfully in 42 msec.
```

Actividad 11 - Función y disparador de auditoría

PgAdmin confirmó `CREATE TRIGGER`. La consulta de `pg_trigger` devolvió el disparador de `clientes` con `tgenabled = O`, por lo que está habilitado para las operaciones normales de la sesión. El disparador invoca la función por cada fila insertada, modificada o eliminada.

Preguntas de la guía

1. **¿Por qué se revoca el acceso a la función para `PUBLIC`?** La revocación restringe el permiso de ejecución que PostgreSQL otorga por defecto. El disparador ya quedó creado por un rol autorizado y puede llamar a la función cuando `app_ventas` modifique `clientes`.

2. **¿Qué pasaría sin `SECURITY DEFINER`?** La hipótesis es que el `UPDATE` de `app_ventas` fallaría al intentar guardar la evidencia, porque la función se ejecutaría con los permisos de ese rol y no tendría acceso de escritura a `auditoria.cambios_clientes`.

3. **¿Quedaron registradas las creaciones?** Sí. `dba.bitacora` mostró `CREATE SCHEMA` en la línea 241, `CREATE TABLE` en la 243, `CREATE OR REPLACE FUNCTION` en la 255 y `CREATE TRIGGER` en la 279. La tabla y el disparador aparecen como continuación de sus respectivos bloques. El nivel `mod` también registra las sentencias de definición de datos.

Captura 69. Disparador habilitado



Captura 70. Registro de las sentencias de creación

```
229  SELECT num, linea
230  FROM dba.bitacora
231  WHERE linea ILIKE '%CREATE SCHEMA auditoria%'
232     OR linea ILIKE '%CREATE TABLE auditoria.cambios_clientes%'
233     OR linea ILIKE '%CREATE OR REPLACE FUNCTION auditoria.registrar_cambio_cliente()%'
234     OR linea ILIKE '%CREATE TRIGGER trg_auditoria_clientes%'
235  ORDER BY num;
```

Data Output | Messages | Notifications

Showing rows: 1 to 4 | Page No: 1 of 1

	num bigint	linea text
1	241	2026-11-08 17:44:35.075 CST [12184] user=postgres db=comercial app=pgAdmin 4 - CONN:7802833 client=::1 LOG: statement: CREATE SCHEMA auditoria;
2	243	CREATE TABLE auditoria.cambios_clientes (
3	255	2026-11-08 17:46:15.581 CST [12184] user=postgres db=comercial app=pgAdmin 4 - CONN:7802833 client=::1 LOG: statement: CREATE OR REPLACE FUNCTION auditoria.registrar_cambio_cli…
4	279	CREATE TRIGGER trg_auditoria_clientes

Actividad 12 - El incidente auditado

Desde PowerShell, app_ventas se conectó a comercial desde 127.0.0.1 con la aplicación app-ventas-web y actualizó a GAMA780912ZZZ el RFC del cliente con id = 3. PostgreSQL devolvió UPDATE 1. En una segunda conexión, el mismo rol intentó consultar auditoria.cambios_clientes y PostgreSQL devolvió permiso denegado al esquema auditoria.

La consulta como postgres mostró el evento 1 de tipo UPDATE para el cliente 3, con app_ventas como usuario de sesión, postgres como usuario efectivo e IP 127.0.0.1. La comparación identificó rfc como el único campo modificado. PgAdmin recortó los valores en la cuadrícula. Las capturas de los comandos y consultas muestran que el valor anterior era GAMA780912XYZ y el nuevo es GAMA780912ZZZ.

Preguntas de la guía

1. **¿Qué pasó con la hipótesis de la Actividad 11?** El cambio quedó auditado porque SECURITY DEFINER permitió que la función escribiera con los permisos de su propietario, postgres. session_user conservó el rol que hizo el cambio, app_ventas.

2. **¿Cómo se restauraría el RFC anterior?** La siguiente sentencia restauraría el valor y generaría un nuevo evento si el disparador está habilitado.

```
UPDATE clientes SET rfc = 'GAMA780912XYZ' WHERE id = 3;
```

3. **¿Qué campo es más confiable para investigar el origen?** ip_cliente describe la dirección que observó el servidor y ayuda a rastrear la conexión. aplicacion aporta contexto, pero el cliente puede elegir ese nombre. Una IP tampoco identifica por sí sola a una persona.

Captura 71. Cambio ejecutado y acceso al historial denegado

```
PS C:\Users\diego> & "C:\Program Files\PostgreSQL\18\bin\psql.exe" -W "host=127.0.0.1 dbname=comercial user=app_ventas a
pplication_name=app-ventas-web" -c "UPDATE clientes SET rfc = 'GAMA780912ZZZ' WHERE id = 3"
Contraseña:

UPDATE 1
PS C:\Users\diego>
PS C:\Users\diego> & "C:\Program Files\PostgreSQL\18\bin\psql.exe" -W "host=127.0.0.1 dbname=comercial user=app_ventas"
-c "SELECT * FROM auditoria.cambios_clientes"
Contraseña:

ERROR:  permiso denegado al esquema auditoria
LÍNEA 1: SELECT * FROM auditoria.cambios_clientes
```

Captura 72. Identidad de sesión y rol efectivo

```
238  SELECT id_evento, momento, usuario_sesion, usuario_efectivo, aplicacion, ip_cliente, operacion, id_cliente
239  FROM auditoria.cambios_clientes ORDER BY id_evento;
```

Data Output | Messages | Notifications

Showing rows: 1 to 1 | Page N

	id_evento [PK] bigint	momento timestamp with time zone	usuario_sesion text	usuario_efectivo text	aplicacion text	ip_cliente inet	operacion text	id_cliente bigint
1	1	2026-11-08 17:48:24.256946-...	app_ventas	postgres	app-ventas-w...	127.0.0.1	UPDATE	3

Captura 73. Comparación del campo RFC

```
241  SELECT a.id_evento, a.momento, a.usuario_sesion, a.ip_cliente,
242         d.key AS campo,
243         a.valores_antes ->> d.key AS antes,
244         d.value #>> '{}' AS despues
245  FROM auditoria.cambios_clientes a
246  CROSS JOIN LATERAL jsonb_each(a.valores_despues) d
247  WHERE a.operacion = 'UPDATE'
248    AND a.valores_antes -> d.key IS DISTINCT FROM d.value
249  ORDER BY a.id_evento;
```

Data Output | Messages | Notifications

Showing rows: 1

	id_evento [PK] bigint	momento timestamp with time zone	usuario_sesion text	ip_cliente inet	campo text	antes text	despues text
1	1	2026-11-08 17:48:24.256946-...	app_ventas	127.0.0.1	rfc	GAMA780912X...	GAMA780912Z...

Actividad 13 - Altas y bajas

PgAdmin devolvió `INSERT 0 1` al crear al cliente de prueba Pedro Ruiz y `DELETE 1` al eliminarlo. En `auditoria.cambios_clientes` permaneció el `UPDATE` anterior como evento `1`. El evento `2` registró el alta del cliente `6` y el evento `3` registró su baja. Ambos quedaron a nombre de `postgres` desde pgAdmin y con IP `::1`.

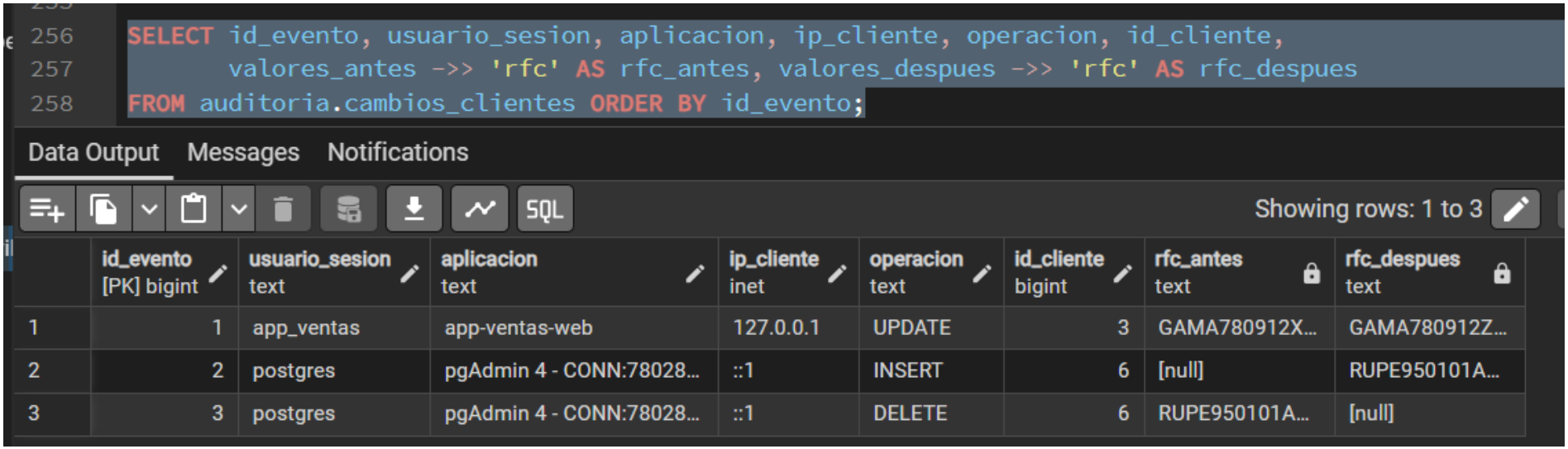
Preguntas de la guía

1. **¿Dónde queda la evidencia de que el cliente existió?** En `auditoria.cambios_clientes`. El `INSERT` guardó la fila en `valores_despues` y el `DELETE` la guardó en `valores_antes`. El otro campo quedó en `NULL` en cada caso porque no existía una fila en ese lado de la operación.

2. **¿Por qué la IP está vacía en el ejemplo de la guía?** Una conexión mediante socket Unix no tiene IP de cliente. En esta práctica pgAdmin se conectó por TCP local mediante IPv6, por eso los eventos muestran `::1`.

3. **¿Qué conflicto existe con la retención?** Borrar una fila de `clientes` deja una copia en la auditoría. El responsable de los datos, en coordinación con las áreas de privacidad y auditoría, debe fijar cuánto tiempo conservarla y cuándo eliminarla según su finalidad y las obligaciones aplicables.

Captura 76. Eventos de modificación, alta y baja



```
256  SELECT id_evento, usuario_sesion, aplicacion, ip_cliente, operacion, id_cliente,
257         valores_antes ->> 'rfc' AS rfc_antes, valores_despues ->> 'rfc' AS rfc_despues
258  FROM auditoria.cambios_clientes ORDER BY id_evento;
```

Data Output | Messages | Notifications

Showing rows: 1 to 3

	id_evento [PK] bigint	usuario_sesion text	aplicacion text	ip_cliente inet	operacion text	id_cliente bigint	rfc_antes text	rfc_despues text
1	1	app_ventas	app-ventas-web	127.0.0.1	UPDATE	3	GAMA780912X...	GAMA780912Z...
2	2	postgres	pgAdmin 4 - CONN:78028...	::1	INSERT	6	[null]	RUPE950101A...
3	3	postgres	pgAdmin 4 - CONN:78028...	::1	DELETE	6	RUPE950101A...	[null]

Actividad 14 - Intentos de manipular la auditoría

Desde PowerShell, `app_ventas` intentó borrar los registros de `auditoria.cambios_clientes` y deshabilitar `trg_auditoria_clientes`. PostgreSQL denegó el primer intento por falta de acceso al esquema `auditoria` y el segundo porque el rol no es dueño de la tabla `clientes`.

Como `postgres`, se deshabilitó el disparador, se cambió a `GAMA780912GHI` el RFC del cliente 3 y se volvió a habilitar el disparador en el mismo bloque. PgAdmin confirmó el último `ALTER TABLE`. El conteo posterior siguió en tres eventos, por lo que esa modificación no generó una fila de auditoría.

La bitácora muestra el intento de `app_ventas` con PID `2012` en las líneas 304 y 306, a las 17:52:42 CST del 8 de noviembre de 2026. El error de propietario aparece en la captura de PowerShell. Las líneas 308 a 310 muestran el bloque ejecutado por `postgres` desde pgAdmin, con PID `12184` e IP `::1`, a las 17:53:27.025 CST. El `UPDATE` y la reactivación del disparador aparecen como continuación del mismo bloque.

Preguntas de la guía

1. **¿Qué evidencia permite detectar el cambio sin auditar?** La bitácora del servidor conserva la secuencia `DISABLE TRIGGER`, `UPDATE` y `ENABLE TRIGGER`. El conteo de la tabla de auditoría siguió en tres, lo que confirma que ese cambio no generó un evento por fila.

2. **¿Qué control administrativo reduciría el riesgo?** Exigir autorización y revisión por una segunda persona para cambios que deshabiliten controles de auditoría. El registro del cambio debe identificar a quien autorizó, a quien ejecutó y a quien verificó la reactivación.

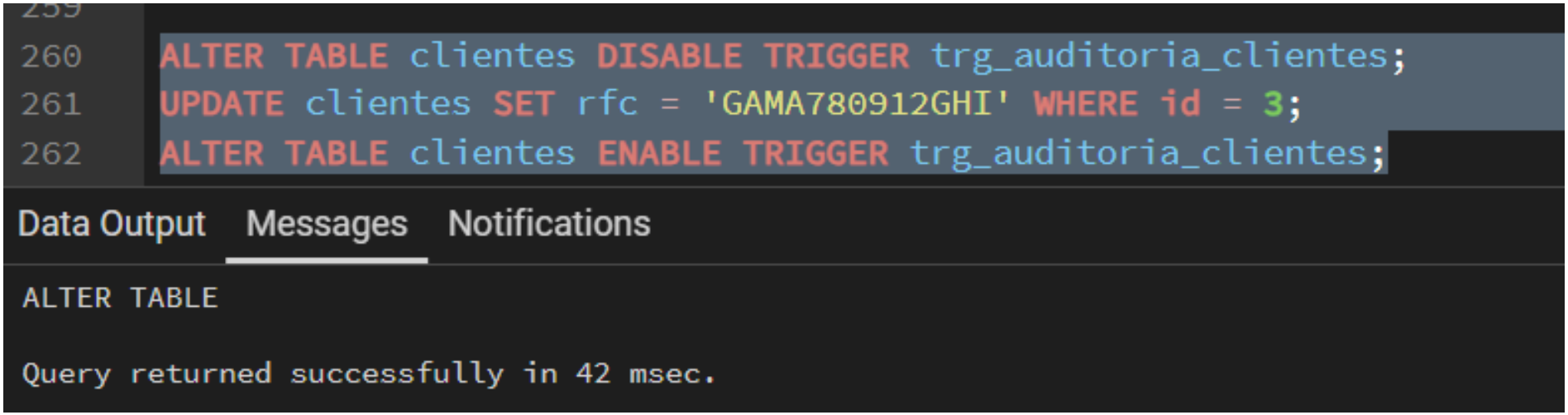
3. **¿Cómo se aplica la separación de funciones?** El DBA administra la base y un equipo distinto custodia y revisa las bitácoras. La revisión independiente permite contrastar la actividad del DBA con la evidencia conservada fuera de su control directo.

Captura 77. Intentos rechazados de app_ventas

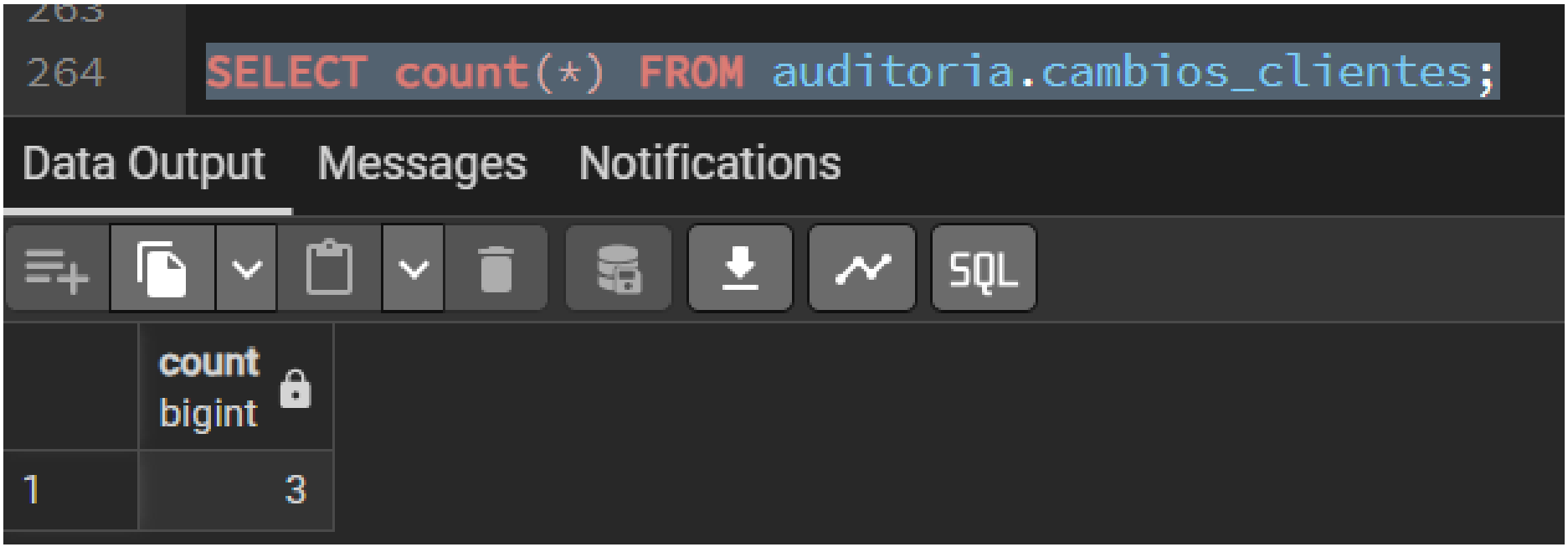
```
PS C:\Users\diego> & "C:\Program Files\PostgreSQL\18\bin\psql.exe" -W "host=127.0.0.1 dbname=comercial user=app_ventas a
pplication_name=app-ventas-web" -c "DELETE FROM auditoria.cambios_clientes" -c "ALTER TABLE clientes DISABLE TRIGGER trg
_auditoria_clientes"
Contraseña:

ERROR:  permiso denegado al esquema auditoria
LÍNEA 1: DELETE FROM auditoria.cambios_clientes
                     ^
ERROR:  debe ser dueño de la tabla clientes
```

Captura 78. Cambio del administrador con el disparador deshabilitado



Captura 79. Conteo de eventos después del cambio



Captura 80. Registro del intento y del bloque del administrador

```
266  SELECT num, linea
267  FROM dba.bitacora
268  WHERE linea ~* 'ALTER TABLE clientes (DISABLE|ENABLE) TRIGGER trg_auditoria_clientes'
269     OR linea LIKE '%UPDATE clientes SET rfc = ''GAMA780912GHI'' WHERE id = 3%'
270  ORDER BY num;
```

Data Output | Messages | Notifications

Showing rows: 1 to 5 | Page No: 1 of 1

	num bigint	linea text
1	304	2026-11-08 17:52:42.221 CST [2012] user=app_ventas db=comercial app=app-ventas-web client=127.0.0.1 LOG: sentencia: ALTER TABLE clientes DISABLE TRIGGER trg_auditoria_clientes
2	306	2026-11-08 17:52:42.222 CST [2012] user=app_ventas db=comercial app=app-ventas-web client=127.0.0.1 SENTENCIA: ALTER TABLE clientes DISABLE TRIGGER trg_auditoria_clientes
3	308	2026-11-08 17:53:27.025 CST [12184] user=postgres db=comercial app=pgAdmin 4 - CONN:7802833 client=::1 LOG: statement: ALTER TABLE clientes DISABLE TRIGGER trg_auditoria_cli…
4	309	UPDATE clientes SET rfc = 'GAMA780912GHI' WHERE id = 3;
5	310	ALTER TABLE clientes ENABLE TRIGGER trg_auditoria_clientes;

Actividad 15 - Límites de la auditoría

Desde PowerShell, `soporte_ana` consultó el nombre, el RFC y el teléfono del cliente 3. `psql` devolvió una fila. En pgAdmin, `postgres` ejecutó `UPDATE clientes SET segmento = 'regular'` y obtuvo `UPDATE 5`.

La bitácora registró una línea para el `UPDATE` masivo. La auditoría por fila mostró seis eventos `UPDATE`, que incluyen el cambio anterior de RFC y los cinco nuevos cambios de segmento, además de un `INSERT` y un `DELETE`. La consulta de lecturas de `soporte_ana` encontró cero coincidencias. Con `log_statement = 'mod'`, el `SELECT` de esta prueba queda fuera del registro de sentencias, y el disparador solo captura cambios de filas.

Preguntas de la guía

1. **¿Qué ocurriría con un millón de filas?** Si el `UPDATE` modifica un millón de filas y se confirma la transacción, el disparador insertaría un millón de eventos de auditoría. Cada evento conservaría los valores anteriores y posteriores. La bitácora de sentencias registraría el `UPDATE` como una sola sentencia.

2. **¿Quién puede autorizar una depuración?** El responsable de los datos puede autorizarla conforme a la política de retención, con revisión del equipo que custodia la evidencia y de las áreas de seguridad y privacidad. El DBA ejecutaría la operación autorizada.

3. **¿Por qué `all` puede acumular información sensible?** Registra el texto de las consultas, que puede incluir RFC, teléfonos u otros valores literales. Esto aumenta el volumen y amplía el conjunto de archivos cuyo acceso debe controlarse.

Captura 81. Lectura de soporte

```
PS C:\Users\diego> & "C:\Program Files\PostgreSQL\18\bin\psql.exe" -W "host=127.0.0.1 dbname=comercial user=soporte_ana"
 -c "SELECT nombre, rfc, telefono FROM clientes WHERE id = 3"
Contraseña:

    nombre    |      rfc       |  telefono
--------------+----------------+------------
 MarÝa GarcÝa | GAMA780912GHI | 6679990000
(1 fila)
```

Captura 82. Actualización de cinco clientes

```
272  UPDATE clientes SET segmento = 'regular';
```

Data Output | Messages | Notifications

```
UPDATE 5

Query returned successfully in 44 msec.
```

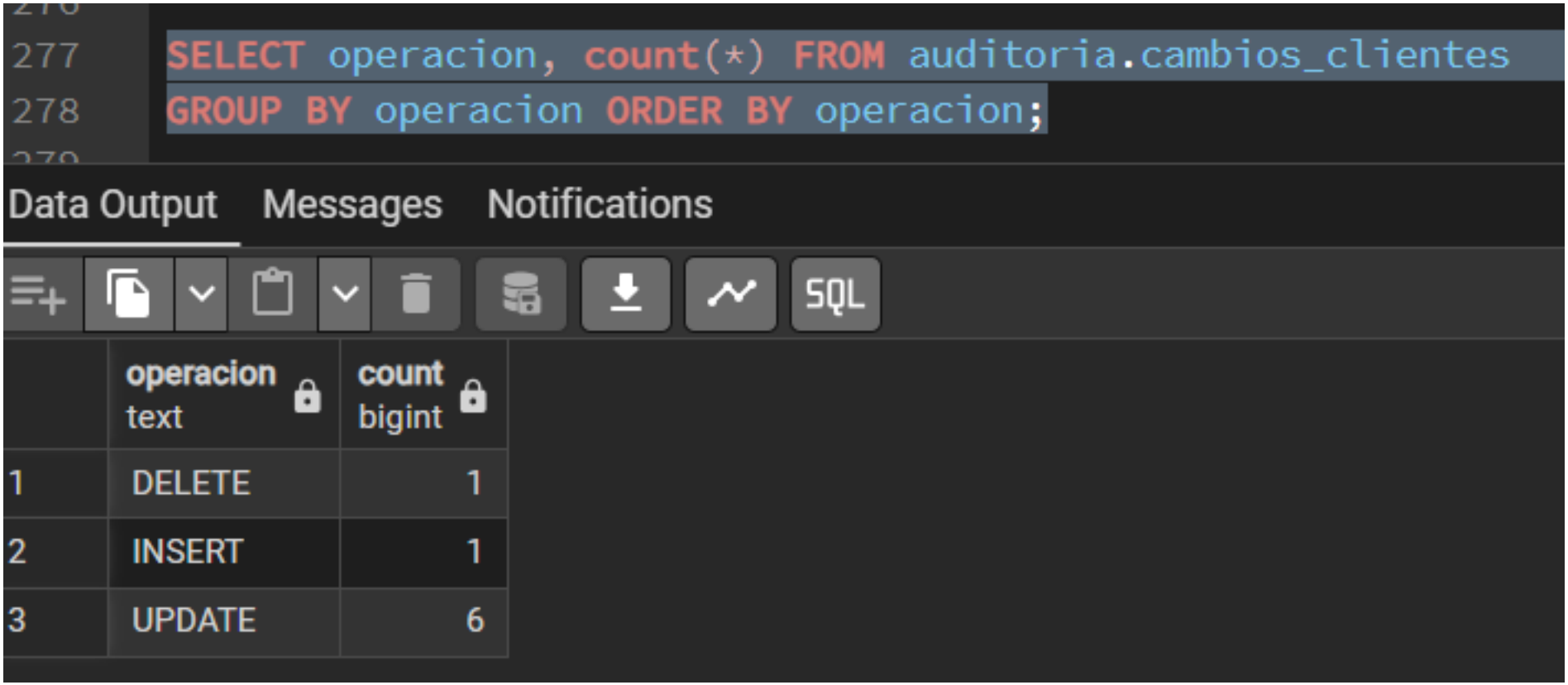
Captura 83. Una sentencia en la bitácora

```
274  SELECT count(*) AS lineas_bitacora FROM dba.bitacora
275  WHERE linea LIKE '%statement: UPDATE clientes SET segmento%';
276
```

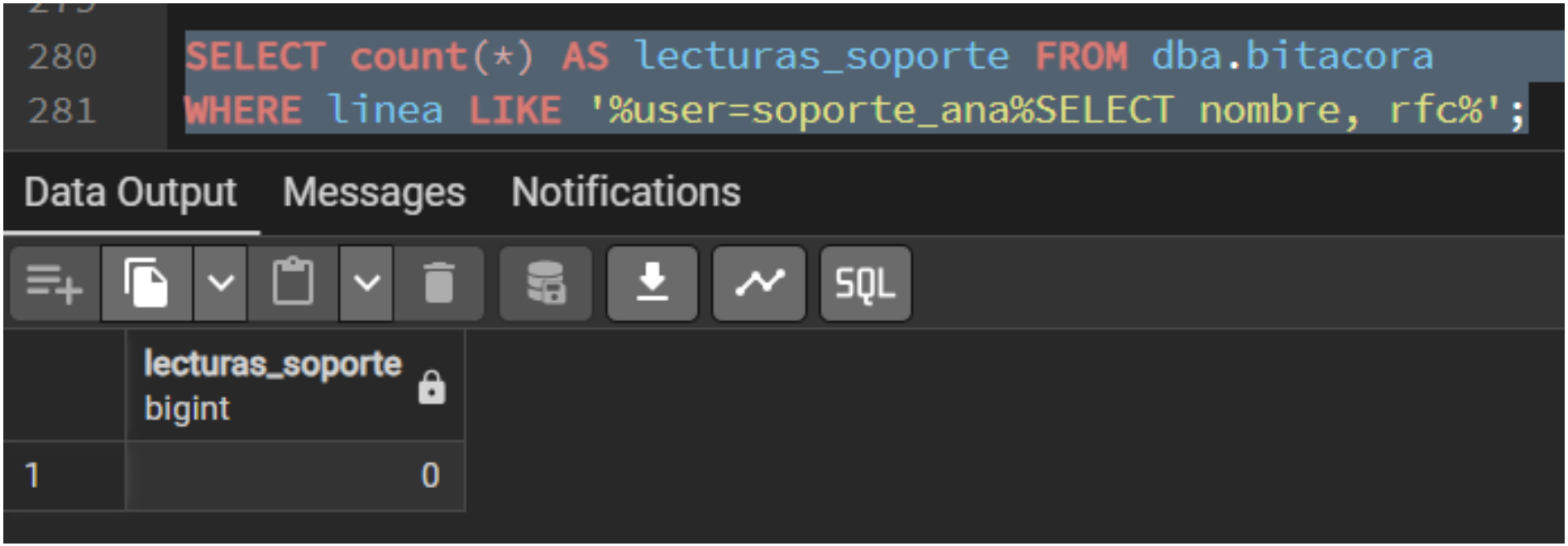
Data Output | Messages | Notifications

	lineas_bitacora bigint
1	1

Captura 84. Eventos en la auditoría por fila



Captura 85. Conteo de lecturas de soporte



Actividad 16 - Exportación y conservación de la bitácora

Se exportó la bitácora vigente desde `comercial` a `evidencias/bitacora-lab01.log`. Para leerla en Windows se utilizó `pg_read_binary_file()` con conversión `WIN1252`. La contraseña de prueba se sustituyó por `<REDACTED>` antes de escribir el archivo. La revisión encontró cuatro reemplazos y ninguna aparición de la contraseña original. El SHA-256 recalculado coincide con el de PowerShell y quedó guardado en `bitacora-lab01.sha256`.

Preguntas de la guía

1. **¿Qué permite demostrar el SHA-256?** Permite comprobar si el archivo conserva los mismos bytes que tenía al calcular un hash de referencia confiable. Si el archivo se modifica, el hash calculado cambia.

2. **¿Dónde se guarda el hash de referencia?** En un repositorio o sistema de evidencias separado, con acceso restringido y control de cambios. Así, quien pueda modificar la bitácora no puede reemplazar también su referencia.

3. **¿Qué riesgo tiene conservar la bitácora solo en el servidor?** Una rotación, un fallo de disco, una reinstalación o una alteración en el servidor puede eliminar la única copia. Una copia separada permite conservar evidencia aunque falle el origen.

4. **¿Cuánto tiempo conservarla?** Para este laboratorio propongo 90 días desde la exportación, suficiente para la revisión y las aclaraciones del curso. Si hay una investigación abierta, se conserva hasta resolverla. Se limita el acceso durante ese plazo.

Captura 87. SHA-256 de la bitácora exportada

```
PS C:\Users\diego\OneDrive\Documents\laboratorios-alexis\UNIDAD 3\LAB-01> Get-FileHash evidencias/bitacora-lab01.log -Al
gorithm SHA256

Algorithm       Hash                                                                   Path
---------       ----                                                                   ----
SHA256          37FD1106454854FC534C009033247649F81E109F6EF02E4FEF7F8D0FE1945D47       C:\Users\diego\OneDrive\Docum...
```

Actividad 17 - Cronología y respuestas a Dirección

Los eventos fechados corresponden al 8 de noviembre de 2026, en CST, según la bitácora exportada. L indica el número de línea del archivo y E el identificador de la fila de auditoría. Las continuaciones de un bloque comparten la hora y el contexto de su primera línea.

#	Hora CST	Usuario	Cliente	Tipo	Qué ocurrió	¿Exitoso?	Fuente
1	17:17:31.243	app_ventas	127.0.0.1	Acceso	Contraseña incorrecta.	No	L105-106
2	17:17:43.484	usuario_inexistente	127.0.0.1	Acceso	Intento con un rol inexistente.	No	L111-112
3	17:17:59.943	app_ventas	127.0.0.1	Acceso	Conexión autorizada para app-ventas-web.	Sí	L119-120
4	17:22:44.710	postgres	::1	Privilegio	REVOKE a soporte y GRANT a ventas.	Sí	L128-129
5	17:22:44.710	postgres	::1	Estructura	Se agregó clientes.segmento.	Sí	L128 y L130
6	17:30:55.552	postgres	::1	Configuración	Se solicitó log_statement = 'mod'. SHOW confirmó el valor después.	Sí	L173 y captura de la Actividad 7
7	17:31:24.803	app_ventas	127.0.0.1	Datos	Actualizó el teléfono del cliente 3.	Sí	L177 y UPDATE 1
8	17:35:09.556	soporte_ana	127.0.0.1	Error	Se rechazó el borrado del cliente 5.	No	L185-186
9	17:37:05.016	postgres	::1	Error	Se rechazó eliminar soporte_ana por dependencias.	No	L215-218
10	17:42:25.817	soporte_ana	127.0.0.1	Acceso	Cerró reporte-mensual tras 90,074 s.	Sí	L238
11	17:48:24.257	app_ventas	127.0.0.1	Datos	Cambió el RFC de María García.	Sí	L285 y E1
12	17:53:27.025	postgres	::1	Datos	Cambió el RFC entre la desactivación y la reactivación del disparador.	Sí	L308-310 y captura de ejecución
13	17:55:43.117	postgres	::1	Datos	Actualizó el segmento de cinco clientes.	Sí	L317 y UPDATE 5

Respuestas a Dirección

1. **¿Quién cambió el RFC?** En las actividades 3 y 7, las capturas de la ejecución controlada identifican a app_ventas. Para atribuir esos cambios a partir de la bitácora del servidor, **No hay evidencia**. En la Actividad 12 el rol de sesión fue app_ventas (L285 y E1). En la Actividad 14 el bloque de postgres quedó en L308-310.

2. **¿Cuándo y desde dónde?** Para la hora exacta de los cambios de las actividades 3 y 7, **No hay evidencia**. Las capturas de PowerShell muestran que el comando usó `app-ventas-web` y `127.0.0.1`. La Actividad 12 quedó registrada a las 17:48:24.257, desde `app-ventas-web` y `127.0.0.1`. La Actividad 14 quedó registrada a las 17:53:27.025, desde pgAdmin y `::1`.

3. **¿Quién cambió el teléfono?** `app_ventas` lo cambió a las 17:31:24.803, desde `app-ventas-web` y `127.0.0.1` (L177). El valor anterior no está en esa sentencia.

4. **¿Hubo intentos de acceso fallidos antes del cambio?** Antes de los cambios auditados aparecen fallos locales desde `127.0.0.1` para `app_ventas` y `usuario_inexistente` (L105 y L111), además de intentos previos de `postgres`. El intento inicial de la Actividad 3 ocurrió antes de activar el registro de conexiones. Para los accesos fallidos anteriores a ese incidente, **No hay evidencia**.

5. **¿Qué RFC había antes de cada cambio?** Las capturas de la práctica muestran `GAMA780912GHI` antes de la Actividad 3 y `GAMA780912XXX` antes de la 7. El evento 1 conserva `GAMA780912XYZ` antes de la 12 y `GAMA780912ZZZ` después. Ese último valor es el último estado observado antes de la 14. En la tabla de auditoría, **No hay evidencia** del valor previo para la 14 porque el disparador estaba deshabilitado.